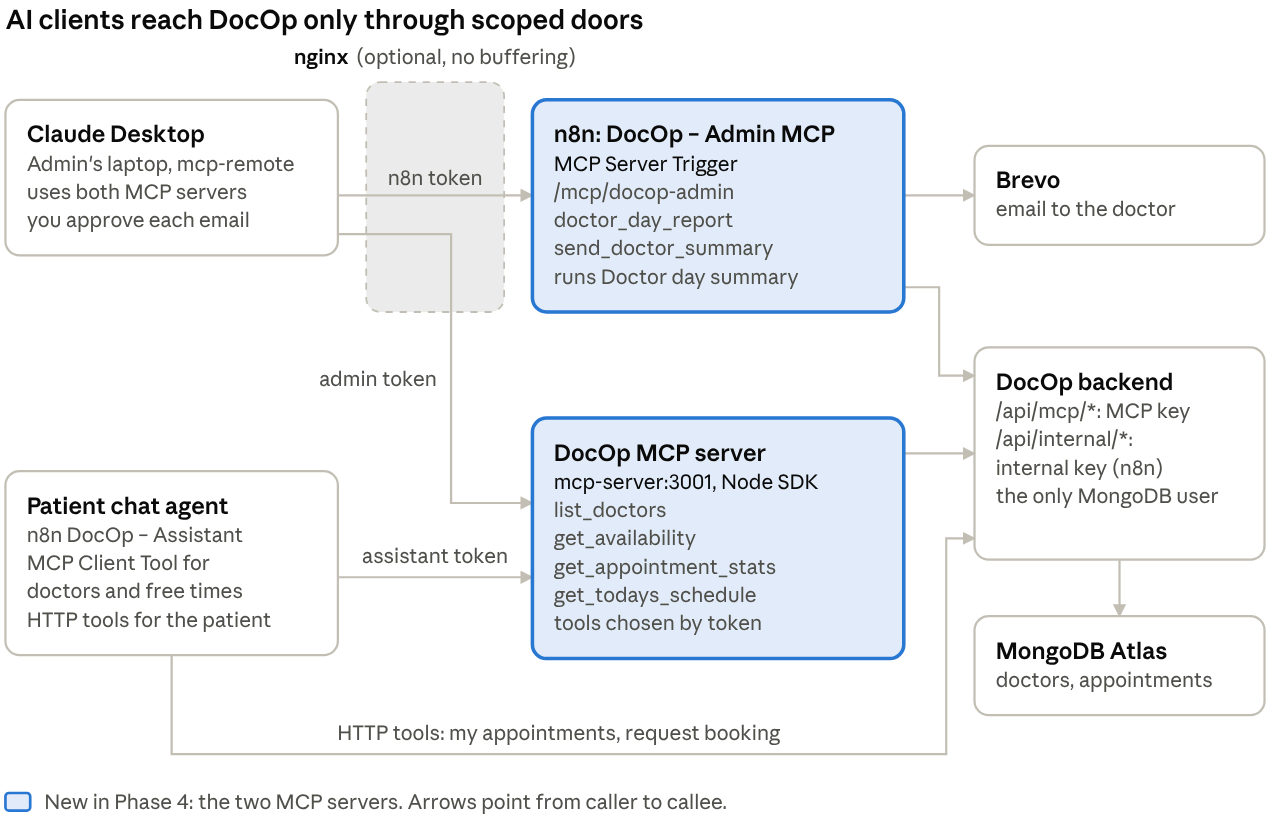
DocOp Phase 4: MCP in Both Directions

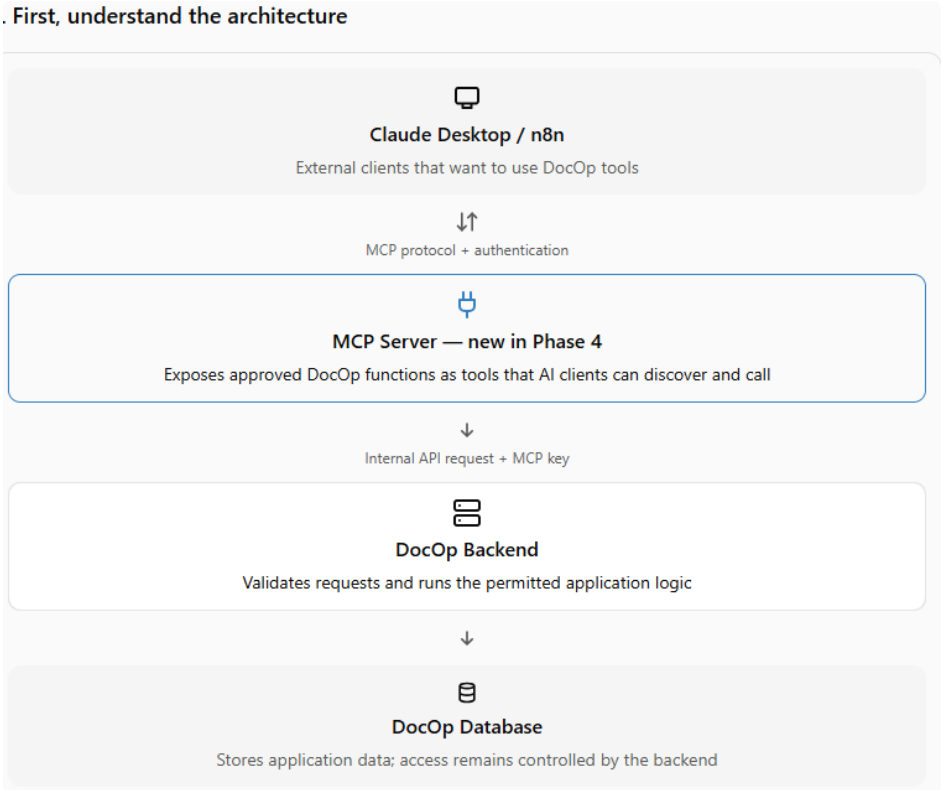
Oct 8, 2026

Architecture

Phase 4 adds **two MCP servers**: one you write (DocOp's data as tools) and one n8n hosts (your workflows as tools). Every AI client reaches DocOp through one of them, with its own token, and only the backend ever touches MongoDB.



Phase 4 architecture · 2 MCP servers, 3 call paths



Think of the MCP server as a secure adapter between AI applications and your DocOp backend.

For example, imagine Claude asks, "Show me the available doctors." The intended flow is:

1. Claude discovers an available DocOp tool through MCP.
2. Claude calls that tool.
3. The MCP server authenticates the request.
4. The MCP server calls an approved backend API endpoint.
5. The backend returns the permitted data.
6. The MCP server sends the result back to Claude.

The MCP server is not the database, and it isn't a replacement for your backend. It gives AI clients a standard way to use selected backend capabilities.

Read it as three paths: Claude Desktop calls both MCP servers; the Phase 3 patient agent calls only the DocOp MCP server (2 tools) plus its own HTTP tools; and both MCP servers call the backend, never the database.

MCP in two sentences. The Model Context Protocol is a standard way for an AI app (the *client*) to discover and call tools offered by a program (the *server*). Write a tool once as an MCP server, and any MCP client can use it: Claude Desktop, n8n, Cursor, your own code.

Part	Role	Why it exists
DocOp MCP server (`mcp-server/`, new)	MCP **server** you write, 4 read-only tools	Makes DocOp usable by any AI client, without hand-wiring HTTP calls in each one
n8n: DocOp – Admin MCP (new)	MCP **server** hosted by n8n (MCP Server Trigger)	Exposes a workflow that *does* something (emails a doctor) as a tool, with n8n's retries, logs and error alerts
Patient chat agent (Phase 3, changed)	MCP **client** (MCP Client Tool node)	Uses the same tools as Claude instead of its own copies
Claude Desktop	MCP **client**	The admin's assistant for the demo
Backend `/api/mcp/*` (new)	Read-only REST routes, own key	Keeps all data rules in the backend; the MCP server holds no database password
nginx (optional)	Reverse proxy	One entry point on a server; must not buffer MCP streams

How the demo question flows: **"How many appointments does Dr. Perera have tomorrow? Send her a summary."**

1. Claude Desktop calls `doctor_day_report` on the n8n MCP server, with `doctorName: "Perera"` and tomorrow's date.
2. n8n runs **DocOp – Doctor day summary**, which asks the backend's internal API for that doctor's day, and returns the count and times (no email address).
3. Claude answers the question, then calls `send_doctor_summary`. Claude Desktop asks the admin to approve.
4. n8n builds the email from database data and sends it through Brevo, the same email path as Phase 2.

Security model

Every hop has its own secret, and each secret opens only what that hop needs. If one leaks, the damage stays small.

Secret (root `.env`)	Who holds it	What it opens	Why it is separate
`MCP_ADMIN_TOKEN`	Claude Desktop (admin's laptop)	All 4 tools on the DocOp MCP server	The "admin token" the plan asks for. Only an admin sees stats and schedules
`MCP_ASSISTANT_TOKEN`	n8n credential for the patient agent	Only `list_doctors` and `get_availability`	Patients talk to this agent. It must never see stats or other patients' schedules
`DOCOP_MCP_KEY`	mcp-server container only	Read-only routes under `/api/mcp/*`	The MCP server can't reach booking, events or emails, even if it's compromised
`N8N_MCP_TOKEN`	Claude Desktop, n8n credential	The n8n MCP Server Trigger	Without it, anyone who finds the URL could email doctors
`DOCOP_INTERNAL_KEY` (Phase 2)	n8n only	`/api/internal/*`, including the doctor email for the summary	Unchanged. Emails never pass through the MCP server or reach Claude

Rules this phase follows:

- **Read-only first.** The DocOp MCP server has no write tools, and its backend routes only use `find` and `aggregate`. Every tool is marked `readOnlyHint: true`.
- **Scopes are enforced in code.** The MCP server registers tools per request, based on which token came in. A patient-agent token simply has no stats tool to call, whatever the prompt says.
- **Minimum data.** Schedules show the patient's first name, time and status. No emails, phone numbers, addresses or medical data go to an AI model.
- **Only one write action, and it is fixed.** "Send her a summary" runs an n8n workflow that builds the email from the database. Claude chooses *which doctor and date*, never the email text or the recipient address.

- **A human approves the write.** Claude Desktop asks the admin before it runs a tool, so the summary email is sent only after a click.
- **Patient tools stay bound to the patient.** `get_my_appointments` and `request_booking` stay as HTTP tools in n8n. Their `userId` comes from the signed request, and an MCP tool's arguments always come from the model.
- **Localhost only for now.** Port 3001 is bound to `127.0.0.1`. When you put it on the internet, it goes behind nginx with HTTPS.
- **Timing-safe token checks**, the same way `authInternal.js` does it.

Generate each new secret with `openssl rand -hex 32` (letters and numbers only, as in Phase 2).

Step 1: Branch, secrets and Docker Compose

Start Phase 4 on its own branch from `AI-Chat-bot`, so Phase 3 stays exactly as it is.

1. **Tag Phase 3 first** (it has no tag yet). In GitHub Desktop: switch to `AI-Chat-bot` → **History** → right-click the latest commit → **Create Tag** → `phase-3-ai-assistant` → **Push origin**.
2. **Create the branch: Current Branch → New Branch** → `phase-4-mcp`, based on `AI-Chat-bot` → **Publish branch**.
3. **Add 4 secrets** to the root `.env` (one `openssl rand -hex 32` each), and the names with empty values to `.env.example`:

```
# Phase 4: MCP
DOCOP_MCP_KEY=
MCP_ADMIN_TOKEN=
MCP_ASSISTANT_TOKEN=
N8N_MCP_TOKEN=
```

`N8N_MCP_TOKEN` isn't read by any container. You paste it into an n8n credential and into Claude Desktop, and keep it in `.env` so you know where it is.

4. **Give the backend the MCP key.** In `docker-compose.yml`, under `backend` → `environment`, add one line:

```
      DOCOP_MCP_KEY: ${DOCOP_MCP_KEY}
```

5. **Replace the placeholder** `# mcp-server: # added in a later phase` with the real service:

```
 mcp-server:
   build: ./mcp-server
   environment:
     PORT: 3001
     DOCOP_API_URL: http://backend:8000
     DOCOP_MCP_KEY: ${DOCOP_MCP_KEY}
     MCP_ADMIN_TOKEN: ${MCP_ADMIN_TOKEN}
     MCP_ASSISTANT_TOKEN: ${MCP_ASSISTANT_TOKEN}
   ports:
     - "127.0.0.1:3001:3001"   # only your own computer can reach it
   depends_on:
     backend:
       condition: service_healthy
   healthcheck:
     test: ["CMD", "wget", "-qO-", "http://localhost:3001/health"]
     interval: 30s
     timeout: 5s
     retries: 3
   restart: unless-stopped
```

Why each line matters:

- `DOCOP_API_URL: http://backend:8000`: inside Docker, the MCP server finds the backend by service name, never `localhost` (same rule as Phase 2).
- `127.0.0.1:3001:3001`: Claude Desktop on your laptop can connect, but other devices on your Wi-Fi can't.
- `depends_on … service_healthy`: the MCP server starts only after `/api/health` answers, so the first tool call doesn't fail.
- The MCP server does **not** get `DOCOP_INTERNAL_KEY` or any database URL. It can only use the read-only routes from Step 2.

If your backend service has no `healthcheck` block, use `depends_on: [backend]` instead.

Step 2: Backend read-only routes

The MCP server never touches MongoDB. It asks the backend, the same way n8n does since Phase 2, so validation and data rules stay in one place. You add 5 files or edits; I ran this code against 23 tests and all pass.

File	New or edit	What it does
`backend/services/dayTools.js`	New	Date helper that **allows past dates** (an admin may ask about yesterday), time sorting, first-name-only
`backend/middleware/authMcp.js`	New	Checks `x-mcp-key`, like `authInternal.js`
`backend/routes/mcpRoute.js`	New	4 read-only routes, one per MCP tool
`backend/server.js`	Edit	Mounts the routes at `/api/mcp`
`backend/routes/internalRoute.js`	Edit	Adds `/doctors/day-summary` for the n8n summary email (Step 5)

Why `/api/mcp` and not `/api/internal/mcp`: `internalRouter` runs `authInternal` for **every** path under `/api/internal`, so a request with the MCP key would be refused before it reached your new router. A separate prefix avoids that trap.

2a. `backend/services/dayTools.js`

`toSlotDate()` from Phase 3 rejects past dates, which is right for booking but wrong for reports. This helper accepts any real date, and defaults to **today in Sri Lanka**.

1. Why do we need `dayTools.js`?

Imagine someone asks your DocOp MCP assistant:

> "Show me all the appointments scheduled for yesterday."

To answer, the system needs to:

1. Understand what date "yesterday" means.
2. Convert that date into the format used by DocOp's appointment records.
3. Find appointments associated with that date.
4. Sort appointments by time.
5. Format the results so the AI assistant can explain them clearly.

You could write this logic inside every route, but that would duplicate code.

Instead, you put the reusable logic into: `backend/services/dayTools.js`

```
//The `dayTools.js` file contains reusable helper functions for DocOp's admin and MCP read-only routes. It validates dates while allowing past dates, defaults to today's date in Sri Lanka, converts dates into the format used by appointment slots, and converts appointment times into minutes for chronological sorting. It also extracts patients' first names to minimize personal data, escapes special characters for safe regex searches, and formats appointment records into a clean list containing the doctor, patient, time, status, and payment indicator.


// Helpers for admin/MCP read routes. Unlike toSlotDate(), these allow past dates,
// because an admin may ask about yesterday's appointments.
const TZ = "Asia/Colombo";

// "2026-10-09" -> { iso: "2026-10-09", slotDate: "9_10_2026" }. No input = today in Sri Lanka.
export function dayKey(input) {
    let iso = String(input || "").trim(); // if a date is provided convert to string otherwise empty string

// if no date was provided use today
    if (!iso) iso = new Intl.DateTimeFormat("en-CA", { timeZone: TZ }).format(new Date()); // en-CA prints YYYY-MM-DD

//This checks whether the input follows YYYY-MM-DD.
    const m = /^(\d{4})-(\d{2})-(\d{2})$/.exec(iso);
    if (!m) return { error: "Use the date format YYYY-MM-DD, for example 2026-10-09." };

//check whether the date actually exists -->This catches impossible calendar dates, such as February 30.
    const [y, mo, d] = m.slice(1).map(Number);
    const dt = new Date(Date.UTC(y, mo - 1, d));
    if (dt.getUTCMonth() !== mo - 1 || dt.getUTCDate() !== d) return { error: `${iso} is not a real date.` };

//return both data formats iso and slotdate
    return { iso, slotDate: `${d}_${mo}_${y}` };
}
```

```
// "02:30 PM" -> 870, so a day's appointments can be sorted by time
//Computers can sort appointment times more reliably if they are converted
into numbers.
export function toMinutes(time) {
    const m = /^(\d{1,2}):(\d{2})\s*(AM|PM)$/i.exec(String(time || ""));
    if (!m) return 24 * 60;
    let h = Number(m[1]) % 12;
    if (m[3].toUpperCase() === "PM") h += 12;
    return h * 60 + Number(m[2]);
}

// Minimum data: only the patient's first name ever leaves the backend for an
AI--This is data minimization
export const firstName = (name) => String(name ||
"Patient").trim().split(/\s+/)[0];

//This helper adds a backslash before special regex characters.-->That makes
it safer to use user-provided text as a literal search pattern.
export const escapeRegex = (s) => s.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");

// One day's appointments for one doctor (or all doctors), sorted by time
//It takes an array of appointment records and produces a cleaner list.
export function toScheduleRows(appointments) {
    return appointments
        .slice()  //Copy the array
//It compares appointment times after converting them to minutes.
        .sort((a, b) => toMinutes(a.slotTime) - toMinutes(b.slotTime))
        .map((a) => ({  //Select only the required fields
            time: a.slotTime,
            doctor: a.docData?.name,
            doctorId: a.docId,
            patient: firstName(a.userData?.name),
            status: a.cancelled ? "cancelled" : a.isCompleted ? "completed" :
"booked",
            paid: Boolean(a.payment),
        }));
}
```

Sorting matters: `slotTime` is text, so sorted as text `"02:30 PM"` comes before `"10:00 AM"`. `toMinutes()` sorts by real time.

2b. `backend/middleware/authMcp.js`

file is responsible for authenticating requests coming from your DocOp MCP server before they can access the backend's MCP routes.

```
import crypto from "crypto";

// Same check as authInternal.js, but with its own header and key.
// Only the mcp-server container has DOCOP_MCP_KEY, and it opens only
/api/mcp/* (read-only).

//Define the authentication middleware-security guard at the entrance to your
MCP API
//  *req — the incoming HTTP request.
//  *res — the HTTP response your backend will send.
//  *next — a function that passes control to the next middleware or route
handler.
export default function authMcp(req, res, next) {

//
    const given = Buffer.from(req.headers["x-mcp-key"] || "");
    const expected = Buffer.from(process.env.DOCOP_MCP_KEY || "");
    if (!expected.length || given.length !== expected.length ||
!crypto.timingSafeEqual(given, expected)) {
        return res.status(401).json({ error: "Unauthorized" });
    }
    next();
}
```

2c. `backend/routes/mcpRoute.js`

One route per MCP tool. Every route only reads. Errors are a short sentence with HTTP 400, so the MCP server can pass them on and the AI can fix its input (the Phase 3 lesson).

```
import express from "express";  //creates API routes
import mongoose from "mongoose"; //validates MongoDB IDs and supports
database queries.
import authMcp from "../middleware/authMcp.js";  //checks whether the request
contains the correct MCP secret key
 //access doctor and appointment records.
import doctorModel from "../models/doctorModel.js";
import appointmentModel from "../models/appointmentModel.js";
```

```
//handle available appointment slots and booking-date validation.
import { allSlotsFor, toSlotDate } from "../services/slotTimes.js";

//validate report dates, safely prepare search patterns, and format
appointment data.
import { dayKey, escapeRegex, toScheduleRows } from
"../services/dayTools.js";

// Read-only data for the DocOp MCP server. Every route only reads (find /
aggregate).
// Errors are short sentences, so the AI can read them and correct itself.
const router = express.Router();

//Every route in this router must pass the MCP authentication middleware
before its handler runs.Requests with a missing or incorrect key are
rejected.
router.use(authMcp); //

const badDoctor = (res) => res.status(400).json({ error: "Unknown doctorId.
Use a doctorId from list_doctors." });

// list_doctors: ?specialty=cardio&onlyAvailable=true
//What happens:
//Reads the requested specialty.
//Escapes special regex characters before searching.
//Filters for available doctors if onlyAvailable=true.
//Retrieves selected doctor fields from MongoDB.
//Limits the results to 25 doctors.
//Returns each doctor's ID, name, specialty, experience, fee, and
availability.
//This route helps the AI discover valid doctor IDs before calling other
tools.

router.get("/doctors", async (req, res) => {
    const filter = {};
    const specialty = String(req.query.specialty || "").trim().slice(0, 50);
    if (specialty) filter.specialization = new RegExp(escapeRegex(specialty),
"i");
    if (req.query.onlyAvailable === "true") filter.availability = true;

    const docs = await doctorModel.find(filter)
        .select("name specialization experience fees
```

```
availability").limit(25).lean();
    res.json({
        count: docs.length,
        doctors: docs.map((d) => ({
            doctorId: String(d._id), name: d.name, specialty:
d.specialization,
            experience: d.experience, fee: d.fees, available:
Boolean(d.availability),
        })),
    });
});

// get_availability: ?doctorId=...&date=2026-10-09 (future dates only, like
the website)

//What happens:
//Checks whether the doctor ID has a valid MongoDB ObjectId format.
//Validates the date using toSlotDate().
//Finds the doctor in MongoDB.
//Checks whether the doctor is available.
//Retrieves already-booked slots for that date.
//Uses allSlotsFor() to generate the day's possible slots.
//Removes booked slots and returns the remaining free slots.
//If the doctor is unavailable, the route returns available: false and an
empty list of free slots


router.get("/availability", async (req, res) => {
    if (!mongoose.isValidObjectId(req.query.doctorId)) return badDoctor(res);
    const { slotDate, error } = toSlotDate(req.query.date);
    if (error) return res.status(400).json({ error });

    const doc = await doctorModel.findById(req.query.doctorId).select("name
availability slots_booked").lean();
    if (!doc) return badDoctor(res);
    if (!doc.availability) return res.json({ doctor: doc.name, date:
req.query.date, available: false, freeSlots: [] });

    const booked = doc.slots_booked?.[slotDate] || [];
    const freeSlots = allSlotsFor(slotDate).filter((t) =>
!booked.includes(t));
    res.json({ doctor: doc.name, date: req.query.date, available: true,
freeCount: freeSlots.length, freeSlots });
```

```
});

// get_appointment_stats: ?date=2026-10-09 (appointments ON that day)
//                     or ?days=7 (bookings MADE in the last N days).
Optional &doctorId=


//What happens:

//Builds a MongoDB filter for the requested period.
//Optionally validates and applies the doctor ID.
//Uses MongoDB's aggregation pipeline to group appointments by doctor.
//Calculates totals, cancelled appointments, completed appointments, and paid
appointments.
//Combines the per-doctor numbers into overall totals.
//Returns the statistics in JSON.

router.get("/stats", async (req, res) => {
    const match = {};
    let period;
    if (req.query.date) {
        const day = dayKey(req.query.date);
        if (day.error) return res.status(400).json({ error: day.error });
        match.slotDate = day.slotDate;
        period = `appointments on ${day.iso}`;
    } else {
        const days = Math.min(Math.max(parseInt(req.query.days, 10) || 7, 1),
90);
        match.date = { $gte: Date.now() - days * 24 * 60 * 60 * 1000 };
        period = `bookings made in the last ${days} days`;
    }
    if (req.query.doctorId) {
        if (!mongoose.isValidObjectId(req.query.doctorId)) return
badDoctor(res);
        match.docId = String(req.query.doctorId);
    }

    const rows = await appointmentModel.aggregate([
        { $match: match },
        {
            $group: {
//$group calculates statistics for multiple doctors in one aggregation query
instead of making separate count queries for every doctor.
```

```
                _id: "$docId",
                doctor: { $first: "$docData.name" },
                total: { $sum: 1 },
                cancelled: { $sum: { $cond: ["$cancelled", 1, 0] } },
                completed: { $sum: { $cond: ["$isCompleted", 1, 0] } },
                paid: { $sum: { $cond: [{ $and: ["$payment", { $not:
["$cancelled"] }] }, 1, 0] } },
            },
        },
        { $sort: { total: -1 } },
    ]);

    const byDoctor = rows.map((r) => ({
        doctorId: r._id, doctor: r.doctor, total: r.total, active: r.total -
r.cancelled,
        cancelled: r.cancelled, completed: r.completed, paid: r.paid,
    }));
    const sum = (key) => byDoctor.reduce((n, r) => n + r[key], 0);
    res.json({
        period,
        totals: { total: sum("total"), active: sum("active"), cancelled:
sum("cancelled"), completed: sum("completed"), paid: sum("paid") },
        byDoctor,
    });
});

// get_todays_schedule: ?date=2026-10-09 (default: today in Sri Lanka)
&doctorId=
//Purpose: Retrieve a day's appointments, optionally for one doctor.
//What happens:
//Validates the date with dayKey(), defaulting to today in Sri Lanka if
omitted.
//Builds a query for appointments scheduled on that date.
//Optionally filters by doctor ID.
//Retrieves only the fields needed for the response.
//Calls toScheduleRows() to sort appointments by time and format their data.
//Returns the date, appointment count, and formatted appointment list.
//The count excludes cancelled appointments, although the returned list still
includes them with their status marked as "cancelled"

router.get("/schedule", async (req, res) => {
    const day = dayKey(req.query.date);
    if (day.error) return res.status(400).json({ error: day.error });
```

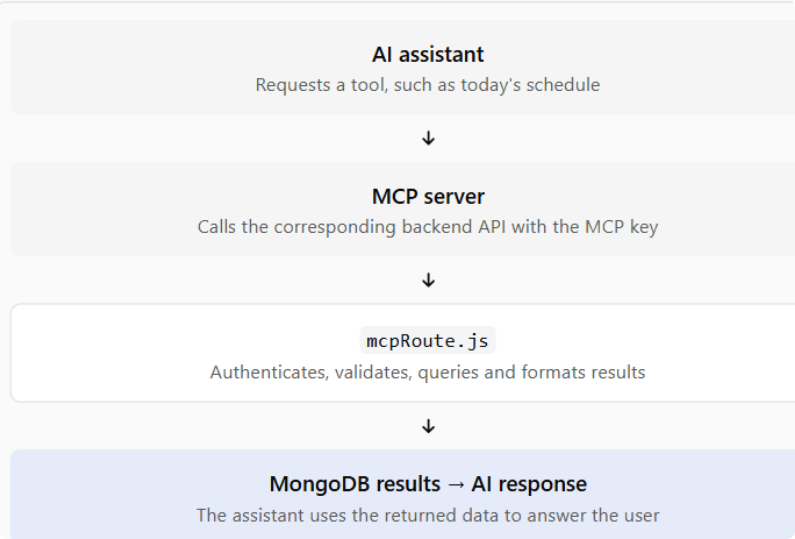
```
    const filter = { slotDate: day.slotDate };
    if (req.query.doctorId) {
        if (!mongoose.isValidObjectId(req.query.doctorId)) return
badDoctor(res);
        filter.docId = String(req.query.doctorId);
    }

    const rows = await appointmentModel.find(filter)
        .select("slotTime docId docData.name userData.name cancelled
isCompleted payment").lean();
    const appointments = toScheduleRows(rows);
    res.json({
        date: day.iso,
        count: appointments.filter((a) => a.status !== "cancelled").length,
        appointments,
    });
});

export default router;
```

Points worth knowing for interviews:

- **`$group` per doctor** gives the admin dashboard numbers in one database round trip, instead of 5 separate `countDocuments` calls.
- **`paid` excludes cancelled** appointments, so refunds don't count as income.
- **`.select(...)`** loads only the fields the response needs. `userData.email` and `userData.phone` are never even read.
- Express 5 (your version) forwards errors from `async` routes automatically, so no `try/catch` is needed
- **Why are these routes read-only?**
  All four routes retrieve or calculate information. They do not create, modify, or cancel appointments.



2d. `backend/server.js`

Next to the other router imports:

```
import mcpRouter from "./routes/mcpRoute.js";
```

Under `app.use("/api/internal/assistant", assistantInternalRouter);`:

```
app.use("/api/mcp", mcpRouter); // Phase 4: read-only data for the MCP server
```

2e. `backend/routes/internalRoute.js` (for Step 5)

This route returns the doctor's **email**, so it stays behind n8n's `DOCOP_INTERNAL_KEY`, not the MCP key. Add two imports at the top:

```
import doctorModel from "../models/doctorModel.js";
import { dayKey, escapeRegex, toScheduleRows } from
"../services/dayTools.js";
```

Then add this route just above `export default router;`:

- This route adds a doctor day-summary feature to your DocOp project. It finds one specific doctor, retrieves their appointments for a selected date, removes cancelled appointments, and returns a summary including the doctor's email so n8n can send the summary email.

- Step-by-step explanation

1. Validate the date — `dayKey()` checks the requested date and converts it into DocOp's appointment-slot format. If no date is provided, it defaults to today in Sri Lanka.
2. Identify the doctor — The request can provide either a `doctorId` or a `doctorName`. If an ID is provided, the code validates it. Otherwise, it searches for a doctor's name, optionally removing the `Dr.` prefix.
3. Handle missing or invalid input — If neither a valid ID nor a name is provided, the route returns HTTP `400 Bad Request`.
4. Find the matching doctor — It searches MongoDB for the doctor and retrieves only their name and email. Name searches are case-insensitive and limited to five matches.
5. Handle ambiguous names — If no doctor matches, it returns `404 Not Found`. If multiple doctors match, it returns `409 Conflict` with their names, allowing the AI assistant to ask which doctor the user means.
6. Retrieve the appointments — Once exactly one doctor is identified, the route finds their appointments for the selected date.
7. Format the schedule — `toScheduleRows()` sorts appointments by time and formats their details. The `.filter()` removes cancelled appointments from the summary.
8. Return the result — The response contains the doctor's ID, name, email, date, appointment count, and appointment list.

```
// Phase 4: one doctor's day, for the n8n "doctor day summary" workflow.
// ?doctorName=Perera or ?doctorId=...  &date=2026-10-09
// Includes the doctor's email, so n8n can send the summary. n8n strips it
before replying to Claude.
router.get("/doctors/day-summary", async (req, res) => {
  const day = dayKey(req.query.date); //validate the date
  if (day.error) return res.status(400).json({ error: day.error });

  let doctors;
  if (req.query.doctorId) {
    if (!mongoose.isValidObjectId(req.query.doctorId)) return
res.status(400).json({ error: "Unknown doctorId." });
    doctors = await doctorModel.find({ _id: req.query.doctorId
}).select("name email").lean();
  } else {
```

```
    const name = String(req.query.doctorName || "").replace(/^dr\.?\s*/i,
"").trim().slice(0, 50);
    if (!name) return res.status(400).json({ error: "Give a doctorName or
doctorId." });
    doctors = await doctorModel.find({ name: new RegExp(escapeRegex(name),
"i") }).select("name email").limit(5).lean();
  }
  if (doctors.length === 0) return res.status(404).json({ error: "No doctor
matches that name." });
  if (doctors.length > 1) {
    return res.status(409).json({ error: "More than one doctor matches. Ask
which one.", matches: doctors.map((d) => d.name) });
  }

  const doc = doctors[0];
  const rows = await appointmentModel.find({ docId: String(doc._id),
slotDate: day.slotDate })
    .select("slotTime docId docData.name userData.name cancelled isCompleted
payment").lean();
  const appointments = toScheduleRows(rows).filter((a) => a.status !==
"cancelled");
  res.json({
    doctorId: String(doc._id), doctorName: doc.name, doctorEmail: doc.email,
    date: day.iso, count: appointments.length, appointments,
  });
});
```

The 409 answer matters for the demo: if two doctors are called Perera, Claude gets their full names back and asks you which one, instead of emailing the wrong doctor.

2f. Tests: `backend/test/mcp.test.js`

23 tests: date helpers, key checks (the MCP key can't open `/api/internal`, and the internal key can't open `/api/mcp`), minimum fields, regex escaping, stats numbers, sorting, first names only, and the name lookup (match, ambiguous, unknown). They use a real in-memory MongoDB, so the `$group` pipeline really runs.

```
import { jest } from '@jest/globals';
import mongoose from 'mongoose';
import { MongoMemoryServer } from 'mongodb-memory-server';

process.env.NODE_ENV = 'test';
```

```
process.env.JWT_SECRET = 'testsecret';
process.env.DOCOP_MCP_KEY = 'test-mcp-key';
process.env.DOCOP_INTERNAL_KEY = 'test-internal-key';

jest.unstable_mockModule('../config/mongodb.js', () => ({ default:
jest.fn(async () => undefined) }));
jest.unstable_mockModule('../config/cloudnary.js', () => ({ default:
jest.fn(() => undefined) }));

const { default: request } = await import('supertest');
const { app } = await import('../server.js');
const { default: doctorModel } = await import('../models/doctorModel.js');
const { default: appointmentModel } = await
import('../models/appointmentModel.js');
const { dayKey, toMinutes, firstName } = await
import('../services/dayTools.js');

const MCP = { 'x-mcp-key': 'test-mcp-key' };
const INTERNAL = { 'x-internal-key': 'test-internal-key' };

// a date 3 days ahead, in both formats
const ahead = new Date(Date.now() + 3 * 864e5);
const iso = new Intl.DateTimeFormat('en-CA', { timeZone: 'Asia/Colombo'
}).format(ahead);
const slot = dayKey(iso).slotDate;

let mongo, perera, silva, off;

const appt = (doc, time, extra = {}) => ({
    userId: new mongoose.Types.ObjectId().toString(), docId: String(doc._id),
slotDate: slot, slotTime: time,
    userData: { name: 'Nimal Fernando', email: 'nimal@test.lk', phone:
'0771234567' },
    docData: { name: doc.name, email: doc.email }, amount: 2000, date:
Date.now(), ...extra,
});

beforeAll(async () => {
    mongo = await MongoMemoryServer.create();
    await mongoose.connect(mongo.getUri());
    const base = { password: 'x', image: 'x', degree: 'MBBS', experience: 5,
phone: '0112345678', about: 'x', address: { line1: 'x' }, date: Date.now() };
    [perera, silva, off] = await doctorModel.create([
```

```
        { ...base, name: 'Dr. Anjali Perera', email: 'perera@test.lk',
specialization: 'Cardiologist', fees: 3000, availability: true, slots_booked:
{ [slot]: ['10:00 AM'] } },
        { ...base, name: 'Dr. Kamal Silva', email: 'silva@test.lk',
specialization: 'Dermatologist', fees: 2500, availability: true,
slots_booked: {} },
        { ...base, name: 'Dr. Off Duty', email: 'off@test.lk',
specialization: 'Cardiologist', fees: 2000, availability: false,
slots_booked: {} },
    ]);
    await appointmentModel.create([
        appt(perera, '02:30 PM', { payment: true }),
        appt(perera, '10:00 AM'),
        appt(perera, '11:00 AM', { cancelled: true }),
        appt(silva, '10:30 AM', { isCompleted: true }),
    ]);
}, 60000);

afterAll(async () => {
    await mongoose.disconnect();
    await mongo?.stop();
});

describe('day helpers', () => {
    test('dayKey converts ISO, allows past, rejects bad dates', () => {
        expect(dayKey('2026-10-09')).toEqual({ iso: '2026-10-09', slotDate:
'9_10_2026' });
        expect(dayKey('2020-01-01').slotDate).toBe('1_1_2020');
        expect(dayKey('2026-02-30').error).toMatch(/not a real date/);
        expect(dayKey('9_10_2026').error).toMatch(/YYYY-MM-DD/);
        expect(dayKey().slotDate).toMatch(/^\d{1,2}_\d{1,2}_\d{4}$/);
    });
    test('toMinutes sorts 12-hour times', () => {
        expect(toMinutes('10:00 AM')).toBe(600);
        expect(toMinutes('12:30 PM')).toBe(750);
        expect(toMinutes('08:30 PM')).toBe(1230);
    });
    test('firstName keeps only the first name', () => {
        expect(firstName('Nimal Fernando')).toBe('Nimal');
        expect(firstName('')).toBe('Patient');
    });
});
```

```
describe('/api/mcp security', () => {
    test.each(['/doctors', '/availability', '/stats', '/schedule'])('%s
refuses a missing or wrong key', async (path) => {
        expect((await request(app).get(`/api/mcp${path}`)).status).toBe(401);
        expect((await request(app).get(`/api/mcp${path}`).set('x-mcp-key',
'wrong')).status).toBe(401);
    });
    test('the internal key does not open MCP routes, and the MCP key does not
open internal routes', async () => {
        expect((await
request(app).get('/api/mcp/doctors').set(INTERNAL)).status).toBe(401);
        expect((await
request(app).get('/api/internal/stats/weekly').set(MCP)).status).toBe(401);
        expect((await request(app).get('/api/internal/doctors/day-summary?
doctorName=Perera').set(MCP)).status).toBe(401);
    });
});

describe('list_doctors', () => {
    test('returns minimum fields, no email or password', async () => {
        const res = await request(app).get('/api/mcp/doctors?
specialty=cardio').set(MCP);
        expect(res.status).toBe(200);
        expect(res.body.count).toBe(2);
        expect(JSON.stringify(res.body)).not.toMatch(/@test\.lk|password/);
        expect(Object.keys(res.body.doctors[0]).sort()).toEqual(['available',
'doctorId', 'experience', 'fee', 'name', 'specialty']);
    });
    test('onlyAvailable hides unavailable doctors', async () => {
        const res = await request(app).get('/api/mcp/doctors?
specialty=cardio&onlyAvailable=true').set(MCP);
        expect(res.body.doctors.map((d) => d.name)).toEqual(['Dr. Anjali
Perera']);
    });
    test('regex input is escaped', async () => {
        const res = await request(app).get('/api/mcp/doctors?
specialty=.*').set(MCP);
        expect(res.body.count).toBe(0);
    });
});

describe('get_availability', () => {
    test('booked slots are removed', async () => {
```

```
        const res = await request(app).get(`/api/mcp/availability?
doctorId=${perera._id}&date=${iso}`).set(MCP);
        expect(res.status).toBe(200);
        expect(res.body.freeSlots).not.toContain('10:00 AM');
        expect(res.body.freeCount).toBe(21);
    });
    test('readable errors for bad doctor or date', async () => {
        const bad = await request(app).get(`/api/mcp/availability?
doctorId=nope&date=${iso}`).set(MCP);
        expect(bad.status).toBe(400);
        expect(bad.body.error).toMatch(/list_doctors/);
        const past = await request(app).get(`/api/mcp/availability?
doctorId=${perera._id}&date=2020-01-01`).set(MCP);
        expect(past.status).toBe(400);
    });
    test('unavailable doctor has no slots', async () => {
        const res = await request(app).get(`/api/mcp/availability?
doctorId=${off._id}&date=${iso}`).set(MCP);
        expect(res.body).toMatchObject({ available: false, freeSlots: [] });
    });
});

describe('get_appointment_stats', () => {
    test('counts per doctor for a day', async () => {
        const res = await request(app).get(`/api/mcp/stats?
date=${iso}`).set(MCP);
        expect(res.status).toBe(200);
        expect(res.body.totals).toEqual({ total: 4, active: 3, cancelled: 1,
completed: 1, paid: 1 });
        const p = res.body.byDoctor.find((r) => r.doctor === 'Dr. Anjali
Perera');
        expect(p).toMatchObject({ total: 3, active: 2, cancelled: 1, paid: 1
});
    });
    test('filters by doctor', async () => {
        const res = await request(app).get(`/api/mcp/stats?
date=${iso}&doctorId=${silva._id}`).set(MCP);
        expect(res.body.totals.total).toBe(1);
    });
    test('last N days by booking time', async () => {
        const res = await request(app).get('/api/mcp/stats?days=7').set(MCP);
        expect(res.body.period).toMatch(/last 7 days/);
        expect(res.body.totals.total).toBe(4);
```

```
    });
    test('bad date is explained', async () => {
        const res = await request(app).get('/api/mcp/stats?
date=tomorrow').set(MCP);
        expect(res.status).toBe(400);
        expect(res.body.error).toMatch(/YYYY-MM-DD/);
    });
});

describe('get_todays_schedule', () => {
    test('sorted by time, first names only, cancelled shown but not counted',
async () => {
        const res = await request(app).get(`/api/mcp/schedule?
date=${iso}&doctorId=${perera._id}`).set(MCP);
        expect(res.status).toBe(200);
        expect(res.body.count).toBe(2);
        expect(res.body.appointments.map((a) => a.time)).toEqual(['10:00 AM',
'11:00 AM', '02:30 PM']);
        expect(res.body.appointments[0].patient).toBe('Nimal');
        expect(JSON.stringify(res.body)).not.toMatch(/Fernando|@|077/);
    });
    test('defaults to today', async () => {
        const res = await request(app).get('/api/mcp/schedule').set(MCP);
        expect(res.status).toBe(200);
        expect(res.body.date).toMatch(/^\d{4}-\d{2}-\d{2}$/);
    });
});

describe('n8n doctor day summary', () => {
    test('finds the doctor by name (with or without "Dr.") and includes the
email for n8n', async () => {
        const res = await request(app).get(`/api/internal/doctors/day-
summary?doctorName=Dr. Perera&date=${iso}`).set(INTERNAL);
        expect(res.status).toBe(200);
        expect(res.body).toMatchObject({ doctorName: 'Dr. Anjali Perera',
doctorEmail: 'perera@test.lk', count: 2 });
        expect(res.body.appointments.map((a) => a.time)).toEqual(['10:00 AM',
'02:30 PM']);
    });
    test('ambiguous name asks which one', async () => {
        const res = await request(app).get(`/api/internal/doctors/day-
summary?doctorName=a&date=${iso}`).set(INTERNAL);
        expect(res.status).toBe(409);
```

```
        expect(res.body.matches.length).toBeGreaterThan(1);
    });
    test('unknown doctor and missing name', async () => {
        expect((await request(app).get(`/api/internal/doctors/day-summary?
doctorName=Nobody&date=${iso}`).set(INTERNAL)).status).toBe(404);
        expect((await request(app).get(`/api/internal/doctors/day-summary?
date=${iso}`).set(INTERNAL)).status).toBe(400);
    });
});
```

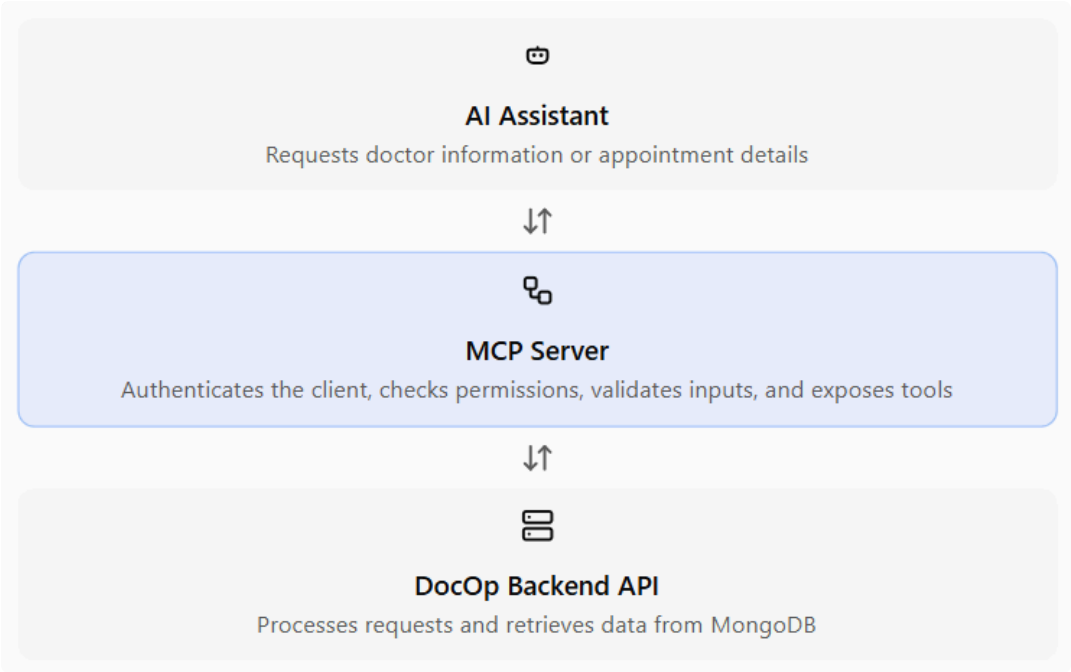
Run it with `cd backend && npm test -- mcp`. The first run downloads MongoDB once (about 100 MB), so it needs internet.

Check before moving on: `docker compose up -d --build backend`, then `docker compose exec backend wget -qO- http://localhost:8000/api/mcp/doctors` should fail with **401**.

Step 3: Build `mcp-server/`

Think of the MCP server as a bridge between an AI assistant and your DocOp backend.

The AI assistant cannot directly access your MongoDB database. Instead, it asks the MCP server to perform a specific task, and the MCP server calls your backend API to get the required information.



A small Node service that speaks MCP over **Streamable HTTP** at `POST /mcp`. I ran it with the official MCP client: 14 of 14 checks pass (auth, scopes, all 4 tools, readable errors).

```
mcp-server/
├── Dockerfile
├── .dockerignore
├── package.json
├── package-lock.json      (created by npm install)
├── scripts/smoke.js       end-to-end check with the official MCP client
└── src/
    ├── index.js           HTTP server, token check, MCP transport
    ├── tools.js           the 4 tools and their input rules
    └── docop.js           calls the backend's /api/mcp routes
```

Key choices, and why:

Choice	Why
Streamable HTTP, not stdio **What is stdio?** • The client starts the MCP server as a local process. • They communicate using standard input and output. • This works well when the client can launch the server directly. **What is Streamable HTTP?** • The MCP server runs as a network service. • The client sends MCP requests over HTTP to an endpoint such as `POST /mcp`. • Other services, including n8n in another Docker container, can communicate with it over the network.	**Why did you choose it?** • My n8n service runs in a separate container, so HTTP makes it possible to communicate with the MCP server using a network address. • For example: `http://mcp-server:3001/mcp` This assumes the containers share a Docker network and the server listens on port `3001` (Step 5).

<table>
<tr><th>Choice</th><th>Why</th></tr>
<tr><td>Stateless (a new server per request, no session ids)
Stateless means the server does not need to maintain a conversation session between requests.
For example:
The AI requests a list of doctors.
The MCP server processes the request and responds.
The AI requests appointment availability in another request.
The server processes that request independently.
The server does not need to store an MCP session ID for this design.</td><td>Nothing to store or clean up. The container can restart mid-demo and clients just carry on.
Why is this useful?
Simpler implementation.
Less session-management code.
Easier container restarts and scaling.
Important: stateless MCP sessions do not mean that the entire application has no state. Your database and AI conversation memory may still retain information.</td></tr>
<tr><td>Tools chosen per token
This is one of the most important security decisions.
Imagine you have two AI assistants:
Patient assistant: can search doctors and check appointment availability.
Admin assistant: can access appointment statistics and daily schedules.
Each assistant receives a different token. The server checks the token and exposes only the tools allowed for that token.</td><td>The patient agent's token never even sees the admin tools. That's stronger than telling the model not to use them.<table><tr><th>Tool</th><th>Patient token</th><th>Admin token</th></tr><tr><td><code>list_doctors</code></td><td>Yes</td><td>Yes</td></tr><tr><td><code>check_availability</code></td><td>Yes</td><td>Yes</td></tr><tr><td><code>get_appointment_stats</code></td><td>No</td><td>Yes</td></tr><tr><td><code>get_daily_schedule</code></td><td>No</td><td>Yes</td></tr></table>These are illustrative permissions; your actual <code>tools.js</code> configuration determines the real access.
Why is this better than simply instructing the AI not to use admin tools?
Because the model's instructions are not a security boundary.
If the server never exposes an admin tool to a patient token, the patient assistant cannot invoke that tool through the MCP interface using that token.
The backend should still enforce its own authentication and authorization.</td></tr>
</table>

Choice	Why
zod input rules • Zod is a JavaScript library used to validate data against defined rules. • Suppose the AI calls `check_availability` with a date. • Valid: `2026-10-15` • Invalid: `next Thursday` • Invalid: `2026-15-40` The tool's Zod schema can reject an incorrectly formatted or impossible date before making a backend request. The same principle applies to doctor IDs and other tool arguments.	A bad date or id is rejected before the backend is called, with a message the AI can act on. **Why is this useful?** • Fewer invalid API requests. • Clearer error messages. • More predictable tool behaviour. Remember: Zod only validates the rules you define. A date can be syntactically valid but still be in the past, so booking-specific restrictions may need additional validation.
Errors as `isError: true`	• The AI reads "Use YYYY-MM-DD" and tries again, the same self-correction as Phase 3. • It is not guaranteed that the AI will always correct itself, but clear errors make that process easier.
`readOnlyHint` on every tool A read-only tool retrieves information but does not intentionally change application data.	Tells MCP clients these tools change nothing. Some clients use this to decide how much to ask before running a tool. **Why is this useful?** • It communicates the tool's intended behaviour to MCP clients. • Some clients may use the hint when deciding how to handle tool calls. • It helps distinguish information retrieval from actions that change data. However, this hint is not a security guarantee. You must ensure the implementation really performs no writes, and the backend must still enforce access controls.

3a. Create the project

In a terminal at the repo root:

```
mkdir -p mcp-server/src mcp-server/scripts
cd mcp-server
npm init -y
npm install @modelcontextprotocol/sdk@1.32.1 express@5.2.1 zod@3.25.76
```

Pinned versions, for the same reason you pinned n8n in Phase 2: a future release can't break your demo. Then replace `package.json` with:

```
{
  "name": "docop-mcp-server",
  "version": "1.0.0",
  "description": "Read-only MCP server that exposes DocOp data as tools",
  "type": "module",
  "main": "src/index.js",
  "scripts": {
    "start": "node src/index.js",
    "smoke": "node scripts/smoke.js"
  },
  "engines": {
    "node": ">=20"
  },
  "dependencies": {
    "@modelcontextprotocol/sdk": "^1.32.1",
    "express": "^5.2.1",
    "zod": "^3.25.76"
  }
}
```

`"type": "module"` lets you use `import`, like your backend.

3b. `src/docop.js`: talking to the backend

- small communication layer between your MCP server and your DocOp backend.
- The main idea is simple: when an AI assistant asks for doctor information, the MCP server uses this file to send an HTTP request to your existing backend. The backend retrieves the data from MongoDB and returns the result.
  Important: The MCP server does not connect directly to MongoDB. It communicates through your backend API.




```
// Small client for the DocOp backend's read-only routes (/api/mcp/*).
// The MCP server never connects to MongoDB itself.
const BASE = process.env.DOCOP_API_URL || "http://backend:8000";

export async function docop(path, params = {})
//path - Which backend endpoint to call
//parms - Optional query parameters {specialty: "cardiology"w}
 {
// build the complete URL -- combines the backend address with the API path
    const url = new URL(`/api/mcp${path}`, BASE);
//Object.entries(params) turns an object into key-value pairs.
//key is the parameter name, such as specialty.
//value is the parameter value, such as cardiology.
//String(value) converts the value to text.
//url.searchParams.set() adds the parameter to the URL.
//It skips values that are undefined, null, or empty strings.

    for (const [key, value] of Object.entries(params)) {
        if (value !== undefined && value !== null && value !== "")
//prevent malformed URLs and unintended query parameters
url.searchParams.set(key, String(value));
    }

//This is the part that actually communicates with your backend.
```

```
    let res;
    try {
        res = await fetch(url, {
            headers: { "x-mcp-key": process.env.DOCOP_MCP_KEY },
            signal: AbortSignal.timeout(10_000), // never hang an AI client
for long
        });
    } catch {
        throw new Error("The DocOp API is not reachable right now. Try again
in a minute.");
    }
//Read the backend response
    const body = await res.json().catch(() => ({}));
    // The backend answers 400 with a readable { error } so the AI can fix
its input

//Handle HTTP errors clearly
    if (!res.ok) throw new Error(body.error || `The DocOp API returned HTTP
${res.status}.`);
    return body;  // Return successful data
}
```

`URLSearchParams` encodes every value, so a specialty like `a&b=c` can't add extra query parameters.

3c. `src/tools.js`: the 4 tools

- The **description** of a tool is what the AI reads to decide when to use it, so each one says what it returns, which date format it wants and where `doctorId` comes from.
- It defines which tools the AI can use, what each tool does, what inputs it accepts, and which tools are available to different types of clients.

Tool / Code block	What it does
`isoDate`	Validates that dates follow the `YYYY-MM-DD` format.
`doctorId`	Validates that a doctor ID has 24 hexadecimal characters.
`READ_ONLY`	Declares that the tools are intended to read data without changing it.

Tool / Code block	What it does
`run(fn)`	Reusable wrapper that executes a tool, returns successful results as JSON text, and handles errors.
`buildServer(scope)`	Creates the DocOp MCP server and registers tools based on the access scope.
`list_doctors`	Retrieves doctors and optionally filters them by specialty. For the assistant scope, it requests only available doctors.
`get_availability`	Retrieves available appointment times for a specified doctor and date.
`if (scope !== "admin") return server`	Prevents non-admin scopes from registering the two admin-only tools.
`get_appointment_stats`	Retrieves appointment statistics, optionally filtered by date, period, or doctor. Admin only.
`get_todays_schedule`	Retrieves a day's appointment schedule, optionally filtered by doctor. Admin only.
`docop(...)`	Calls the relevant DocOp backend API endpoint to retrieve data.
`return server`	Returns the configured MCP server with the tools permitted for that scope.

1. What is `tools.js` responsible for?

Imagine a user asks your DocOp AI assistant:

> "Find a cardiologist and tell me their available appointment times tomorrow."

The AI needs to perform two tasks:

1. Search for doctors.
2. Check the availability of a selected doctor.

Your `tools.js` file gives the AI access to tools that can perform those tasks.

The important distinction is:

- `tools.js` defines **what the AI can do**.
- `docop.js` handles **how the MCP server communicates with your backend**.
- Your backend handles **how the data is retrieved**.

1. readOnlyHint: true -->The tool is intended to read information without changing data.
2. destructiveHint: false -->The tool is not intended to perform destructive actions.
3. idempotentHint: true-->Repeating the same request should not change application state.
4. openWorldHint: false-->The tool is not intended to interact with arbitrary external systems or open-ended outside resources.

6. Step 5 — Build the MCP server based on scope

JavaScript

```
export function buildServer(scope) {
    const server = new McpServer({
        name: "docop",
        version: "1.0.0"
    });
```

This function creates the MCP server.

- `buildServer(scope)` receives a scope such as `"admin"` or `"assistant"`.
- `new McpServer(...)` creates the server instance.
- `name` identifies the server as `docop`.
- `version` identifies the server version.

The important idea is that **the available tools depend on the scope supplied to this function.**

Your `index.js` is expected to authenticate the client, determine its permitted scope from its token, and call `buildServer(scope)` accordingly.

For example:

- An admin token → `buildServer("admin")`
- An assistant token → `buildServer("assistant")`

The scope must come from the server's trusted authentication configuration, not from a value the client can freely choose in its request.

why the description is important to a tool

Why is the description important?

The AI reads the description to understand when and how to use the tool.

It explains:

- What information the tool returns.
- Which filter can be used.
- That the ID returned can be passed to other tools.

A clear description helps the AI choose the right tool and supply suitable arguments.

import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { z } from "zod";
import { docop } from "./docop.js"; //reusable API client you studied in the previous step.

The assistant scope also forces `onlyAvailable=true`, so a patient is never offered a doctor who isn't taking bookings, matching Phase 3's `search_doctors`.

```
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { z } from "zod";
import { docop } from "./docop.js";

// 1. Validate tool inputs
const isoDate = z
    .string()
    .regex(
        /^\d{4}-\d{2}-\d{2}$/,
        "Use YYYY-MM-DD, for example 2026-10-09"
    );

const doctorId = z
    .string()
    .regex(
        /^[a-f0-9]{24}$/i,
        "Use a doctorId returned by list_doctors"
    );

// 2. Declare that tools are read-only
const READ_ONLY = {
    readOnlyHint: true,
    destructiveHint: false,
    idempotentHint: true,
    openWorldHint: false,
};

// 3. Reusable tool execution and error-handling wrapper
const run = (fn) => async (args) => {
    try {
        const data = await fn(args);

        return {
            content: [
                {
                    type: "text",
```

```
                    text: JSON.stringify(data),
                },
            ],
        };
    } catch (err) {
        return {
            isError: true,
            content: [
                {
                    type: "text",
                    text: err.message || "An unexpected error occurred.",
                },
            ],
        };
    }
};

// 4. Build the MCP server according to the authenticated scope
export function buildServer(scope) {
    const server = new McpServer({
        name: "docop",
        version: "1.0.0",
    });

    // Tool 1: List doctors
    server.registerTool(
        "list_doctors",
        {
            title: "List doctors",
            description:
                "List DocOp doctors with their doctorId, specialty, years of
experience, fee (LKR), and booking availability. " +
                "Optionally filter by specialty, e.g. 'cardio' or
'Dermatologist'. " +
                "Use the doctorId with the other tools.",
            inputSchema: {
                specialty: z
                    .string()
                    .max(50)
                    .optional()
                    .describe(
                        "Part of a specialty name. Leave empty for all
doctors."
```

```
                    ),
            },
            annotations: READ_ONLY,
        },
        run(({ specialty }) =>
            docop("/doctors", {
                specialty,
                onlyAvailable:
                    scope === "assistant" ? "true" : undefined,
            })
        )
    );

    // Tool 2: Get appointment availability
    server.registerTool(
        "get_availability",
        {
            title: "Get free appointment times",
            description:
                "Get free 30-minute appointment times in Sri Lanka time for
one doctor on a future date.",
            inputSchema: {
                doctorId: doctorId.describe(
                    "doctorId returned by list_doctors"
                ),
                date: isoDate.describe(
                    "Date in YYYY-MM-DD format. Must be today or later."
                ),
            },
            annotations: READ_ONLY,
        },
        run(({ doctorId, date }) =>
            docop("/availability", { doctorId, date })
        )
    );

    // Patient-facing scope receives only the first two tools
    if (scope !== "admin") {
        return server;
    }

    // Tool 3: Get appointment statistics (admin only)
    server.registerTool(
```

```
        "get_appointment_stats",
        {
            title: "Appointment statistics",
            description:
                "Get appointment counts, including total, active, cancelled,
completed, and paid appointments. " +
                "Optionally filter by date, booking period, or doctor.",
            inputSchema: {
                date: isoDate
                    .optional()
                    .describe("Appointments on this day, YYYY-MM-DD."),
                days: z
                    .number()
                    .int()
                    .min(1)
                    .max(90)
                    .optional()
                    .describe(
                        "Number of days for the booking-period query, from 1
to 90."
                    ),
                doctorId: doctorId
                    .optional()
                    .describe(
                        "Filter by a doctorId returned by list_doctors."
                    ),
            },
            annotations: READ_ONLY,
        },
        run(({ date, days, doctorId }) =>
            docop("/stats", { date, days, doctorId })
        )
    );

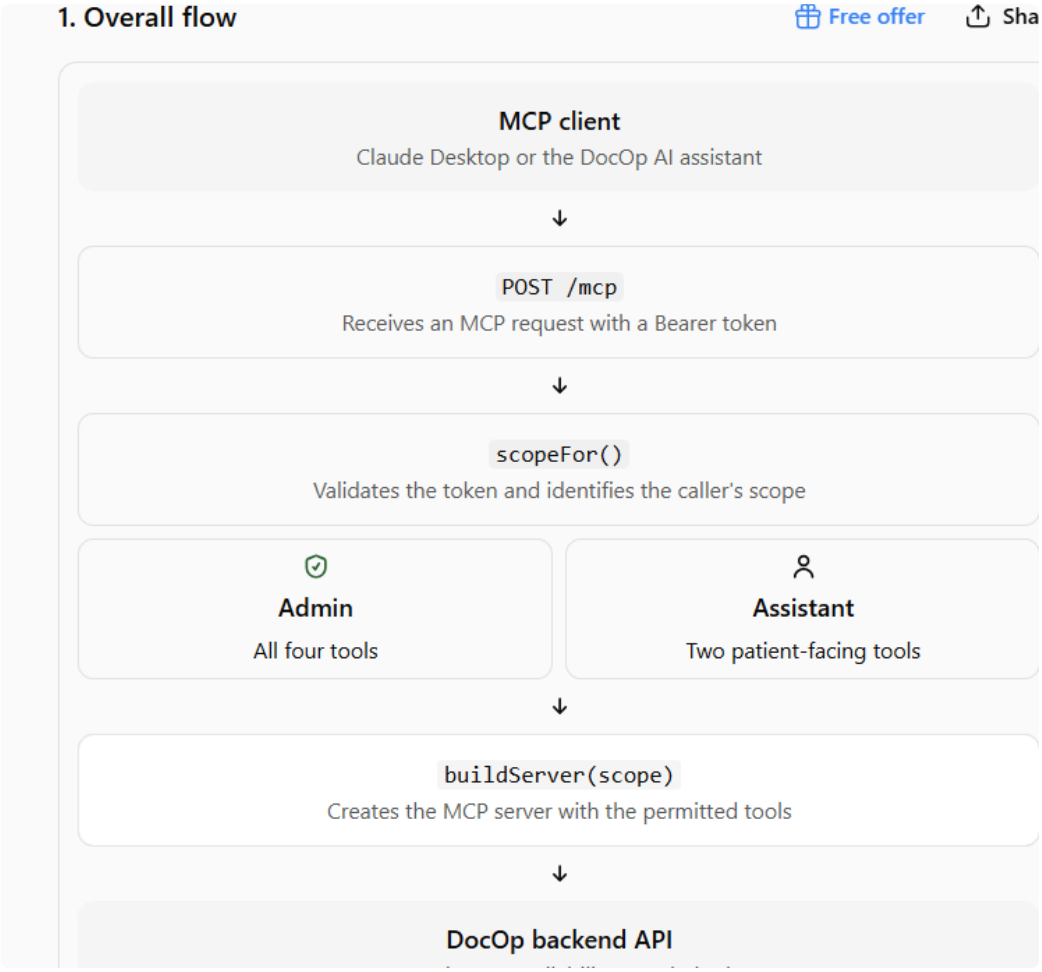
    // Tool 4: Get a daily appointment schedule (admin only)
    server.registerTool(
        "get_todays_schedule",
        {
            title: "Day schedule",
            description:
                "Get appointments for a day, sorted by time, including
doctor, patient first name, status, and payment information. " +
                "Defaults to today in Sri Lanka time. Optionally filter by
```

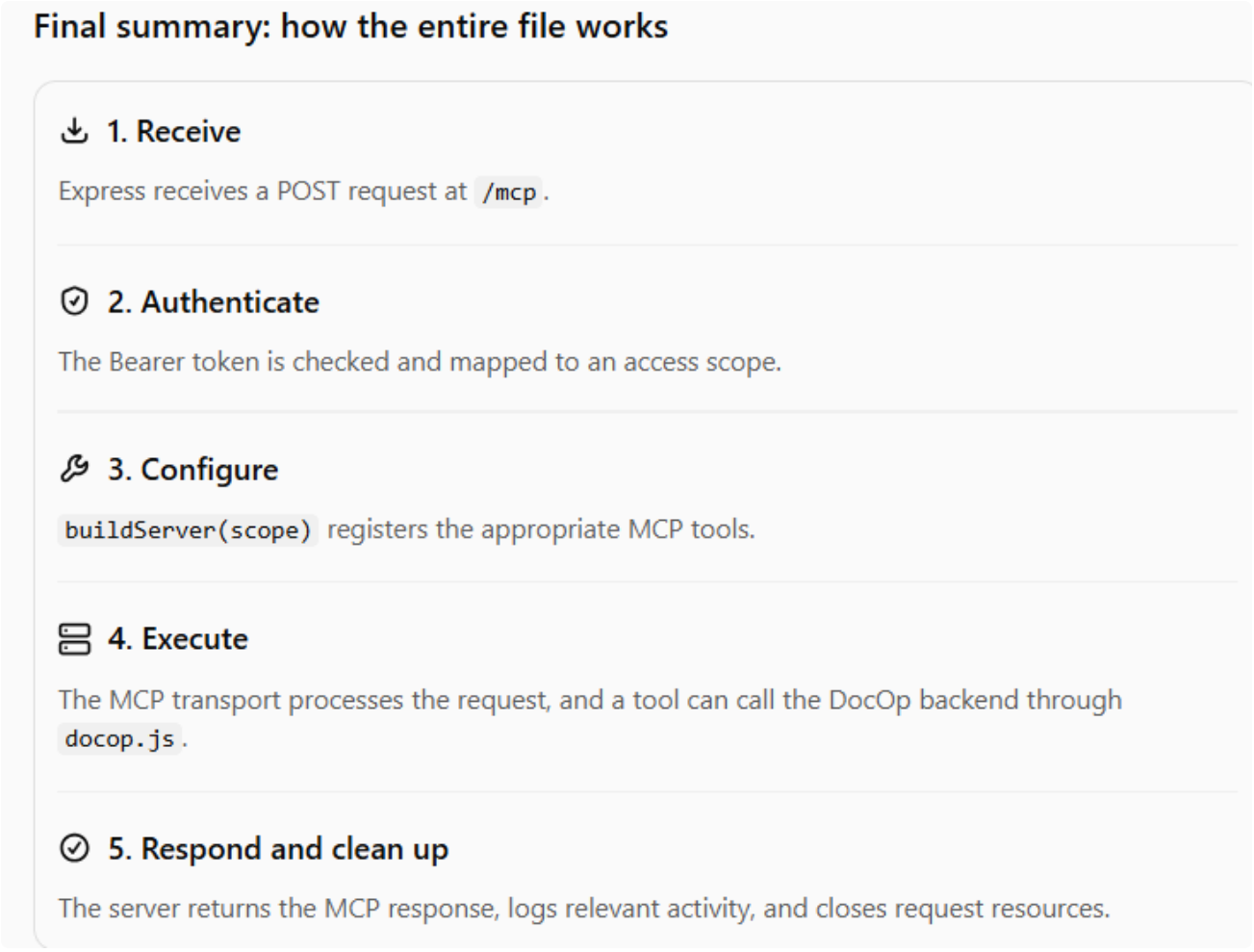
```
doctor.",
            inputSchema: {
                date: isoDate
                    .optional()
                    .describe(
                        "Date in YYYY-MM-DD format. Leave empty for today."
                    ),
                doctorId: doctorId
                    .optional()
                    .describe(
                        "Filter by a doctorId returned by list_doctors."
                    ),
            },
            annotations: READ_ONLY,
        },
        run(({ date, doctorId }) =>
            docop("/schedule", { date, doctorId })
        )
    );

    return server;
}
```

3d. `src/index.js` : HTTP, auth and transport

- Its job is to receive MCP requests, authenticate the caller, decide which tools they can access, and execute those requests.


1. Overall flow
Free offer
Sha
MCP client
Claude Desktop or the DocOp AI assistant
POST /mcp
Receives an MCP request with a Bearer token
scopeFor()
Validates the token and identifies the caller's scope
Admin
All four tools
Assistant
Two patient-facing tools
buildServer(scope)
Creates the MCP server with the permitted tools
DocOp backend API



DocOp MCP Server – Implementation Explanation

1. **Set up the MCP server**
   - Used **Express.js** to create an HTTP server.
   - Integrated the official MCP SDK's Streamable HTTP transport to communicate with MCP clients.
   - Set the server port through environment variables, with port `3001` as the default.
2. **Configured environment-based security**
   - Stored authentication tokens and backend API secrets in environment variables.
   - Added a startup check to prevent the server from running if essential secrets are missing.
3. **Implemented token-based authentication**
   - Created separate authentication tokens for admin and assistant clients.
   - Used Bearer token authentication to identify incoming requests.
   - Used `crypto.timingSafeEqual()` to compare tokens safely.

4. **Implemented scope-based access control**
   - Mapped authenticated tokens to `admin` or `assistant` scopes.
   - Passed the authenticated scope to `buildServer()` in `tools.js`.
   - Admin clients can access all four tools, while assistant clients can access only doctor search and appointment availability.
5. **Created the MCP request endpoint**
   - Implemented `POST /mcp` to receive MCP protocol messages.
   - Rejected requests with missing or invalid tokens using HTTP `401 Unauthorized`.
   - Connected the MCP server to the Streamable HTTP transport to process incoming requests.
6. **Integrated the MCP tools**
   - Connected `index.js` to `tools.js`, where I defined four read-only tools:
     - `list_doctors`
     - `get_availability`
     - `get_appointment_stats`
     - `get_todays_schedule`
   - These tools retrieve data through the DocOp backend API.
7. **Separated client authentication from backend authentication**
   - Used MCP access tokens to authenticate clients connecting to the MCP server.
   - Used a separate `DOCOP_MCP_KEY` when the MCP server calls the DocOp backend.
   - This separates client access control from service-to-service authentication.
8. **Added error handling and monitoring**
   - Created a reusable JSON-RPC error response function.
   - Added error handling for failed MCP request processing.
   - Logged tool calls for debugging and monitoring.
   - Added a `/health` endpoint for basic health checks.
9. **Configured stateless request handling**
   - Configured the Streamable HTTP transport without generated session IDs.
   - Created a server and transport for each POST request and added cleanup logic.
   - This avoids maintaining persistent MCP sessions in this implementation.

```
import crypto from "node:crypto";
import express from "express";
import { StreamableHTTPServerTransport } from
"@modelcontextprotocol/sdk/server/streamableHttp.js";
import { buildServer } from "./tools.js";

const PORT = Number(process.env.PORT || 3001);

// Refuse to start without the secrets: a server with no token would be open
to anyone
//Checking that required secrets exist
for (const name of ["DOCOP_MCP_KEY", "MCP_ADMIN_TOKEN"]) {
    if (!process.env[name]) {
        console.error(`[mcp] ${name} is not set. Add it to the root .env and
restart.`);
        process.exit(1);
    }
}

// Each token maps to a scope. The scope decides which tools exist for that
caller.
const TOKENS = [
    { token: process.env.MCP_ADMIN_TOKEN, scope: "admin" },
    { token: process.env.MCP_ASSISTANT_TOKEN, scope: "assistant" },
].filter((t) => t.token);

// "Authorization: Bearer <token>" -> "admin" | "assistant" | null (timing-
safe, like authInternal.js)
//The scopeFor() authentication function
function scopeFor(header) {
    const match = /^Bearer\s+(.+)$/i.exec(header || "");
    if (!match) return null;
    const given = Buffer.from(match[1].trim());
    for (const { token, scope } of TOKENS) {
        const expected = Buffer.from(token);
        if (given.length === expected.length && crypto.timingSafeEqual(given,
expected)) return scope;
    }
    return null;
}

const rpcError = (res, status, code, message) =>
    res.status(status).json({ jsonrpc: "2.0", error: { code, message }, id:
```

```
null });

const app = express();
app.use(express.json({ limit: "100kb" }));

// For the Docker healthcheck. Says nothing about the data.
app.get("/health", (req, res) => res.json({ status: "ok" }));

// Streamable HTTP, stateless: every POST gets a fresh server + transport.
// No sessions to store, so the container can restart (or scale) without
breaking clients.
app.post("/mcp", async (req, res) => {
    const scope = scopeFor(req.headers.authorization);
    if (!scope) return rpcError(res, 401, -32001, "Unauthorized");

    res.setHeader("X-Accel-Buffering", "no"); // tells nginx not to buffer
this response (Step 6)

    const server = buildServer(scope);
    const transport = new StreamableHTTPServerTransport({ sessionIdGenerator:
undefined });
    res.on("close", () => {
        transport.close();
        server.close();
    });

    try {
        await server.connect(transport);
        await transport.handleRequest(req, res, req.body);
        if (req.body?.method === "tools/call") console.log(`[mcp] ${scope}
called ${req.body.params?.name}`);
    } catch (err) {
        console.error("[mcp] request failed:", err);
        if (!res.headersSent) rpcError(res, 500, -32603, "Internal server
error");
    }
});

// Stateless mode has no server-to-client stream and no sessions to delete
app.get("/mcp", (req, res) => rpcError(res, 405, -32000, "Method not
allowed"));
app.delete("/mcp", (req, res) => rpcError(res, 405, -32000, "Method not
allowed"));
```

```
app.listen(PORT, () => {
    console.log(`[mcp] DocOp MCP server on port ${PORT}, scopes:
${TOKENS.map((t) => t.scope).join(", ")}`);
});
```

The log line `[mcp] admin called get_todays_schedule` gives you an audit trail: who (which scope) used which tool. It never logs arguments, so no patient data ends up in logs.

3e. `Dockerfile` and `.dockerignore`

Same pattern as your backend image: small, production-only, not root.

```
FROM node:20-alpine
WORKDIR /app
ENV NODE_ENV=production

# Install only what's needed to run (no dev tools), from the lock file
COPY package*.json ./
RUN npm ci --omit=dev && npm cache clean --force

COPY src ./src

# Don't run as root (same as the backend image)
USER node
EXPOSE 3001
CMD ["node", "src/index.js"]
```

`.dockerignore`:

```
node_modules
scripts
.env
npm-debug.log
```

3f. `scripts/smoke.js`: prove it works

This uses the **official MCP client**, so it tests exactly what n8n and Claude Desktop will do. It runs on your laptop, not in the container.

```
// Smoke test for a running DocOp MCP server, using the official MCP client.
// Usage (from mcp-server/):
//   MCP_ADMIN_TOKEN=... MCP_ASSISTANT_TOKEN=... npm run smoke
// Optional: MCP_URL (default http://localhost:3001/mcp)
import { Client } from "@modelcontextprotocol/sdk/client/index.js";
import { StreamableHTTPClientTransport } from
"@modelcontextprotocol/sdk/client/streamableHttp.js";

const URL_ = new URL(process.env.MCP_URL || "http://localhost:3001/mcp");
let pass = 0, fail = 0;
const check = (ok, label, detail = "") => {
    ok ? pass++ : fail++;
    console.log(`${ok ? "PASS" : "FAIL"}  ${label}${detail ? `  (${detail})`
: ""}`);
};

async function connect(token) {
    const client = new Client({ name: "docop-smoke", version: "1.0.0" });
    const headers = token ? { Authorization: `Bearer ${token}` } : {};
    await client.connect(new StreamableHTTPClientTransport(URL_, {
requestInit: { headers } }));
    return client;
}
const json = (result) => JSON.parse(result.content[0].text);

// 1. No token / wrong token
for (const [label, token] of [["no token", ""], ["wrong token", "not-a-real-
token"]]) {
    try {
        await connect(token);
        check(false, `${label} is refused`, "connected anyway");
    } catch (err) {
        check(/401|Unauthorized/i.test(String(err)), `${label} is refused`,
String(err.message).slice(0, 60));
    }
}

// 2. Admin token: 4 read-only tools
const admin = await connect(process.env.MCP_ADMIN_TOKEN);
const adminTools = (await admin.listTools()).tools;
check(adminTools.length === 4, "admin sees 4 tools", adminTools.map((t) =>
t.name).join(", "));
check(adminTools.every((t) => t.annotations?.readOnlyHint === true), "every
```

```
tool is marked read-only");

const doctors = json(await admin.callTool({ name: "list_doctors", arguments:
{} }));
check(Array.isArray(doctors.doctors), "list_doctors works", `${doctors.count}
doctors`);
check(!JSON.stringify(doctors).includes("@"), "list_doctors returns no email
addresses");

const today = json(await admin.callTool({ name: "get_todays_schedule",
arguments: {} }));
check(typeof today.count === "number", "get_todays_schedule works",
`${today.date}: ${today.count} appointments`);

const stats = json(await admin.callTool({ name: "get_appointment_stats",
arguments: { days: 7 } }));
check(typeof stats.totals?.total === "number", "get_appointment_stats works",
`${stats.period}: ${stats.totals?.total}`);

if (doctors.doctors?.[0]) {
    const tomorrow = new Date(Date.now() + 864e5).toLocaleDateString("en-CA",
{ timeZone: "Asia/Colombo" });
    const free = json(await admin.callTool({ name: "get_availability",
arguments: { doctorId: doctors.doctors[0].doctorId, date: tomorrow } }));
    check(Array.isArray(free.freeSlots), "get_availability works",
`${free.doctor} on ${tomorrow}: ${free.freeSlots?.length} free`);
}

// 3. Bad input comes back as a readable tool error, not a crash
const bad = await admin.callTool({ name: "get_todays_schedule", arguments: {
date: "tomorrow" } });
check(bad.isError === true, "a bad date is a readable tool error",
bad.content?.[0]?.text?.slice(0, 60));
const past = await admin.callTool({ name: "get_availability", arguments: {
doctorId: "0".repeat(24), date: "2020-01-01" } });
check(past.isError === true, "a past date for availability is refused",
past.content?.[0]?.text?.slice(0, 60));
await admin.close();

// 4. Assistant token: only the 2 patient-safe tools
if (process.env.MCP_ASSISTANT_TOKEN) {
    const assistant = await connect(process.env.MCP_ASSISTANT_TOKEN);
    const names = (await assistant.listTools()).tools.map((t) =>
```

```
t.name).sort();
    check(names.join() === "get_availability,list_doctors", "assistant sees
only list_doctors and get_availability", names.join(", "));
    const hidden = await assistant.callTool({ name: "get_appointment_stats",
arguments: {} }).catch((e) => ({ isError: true, content: [{ text: e.message
}] }));
    check(hidden.isError === true, "assistant can't call
get_appointment_stats");
    const avail = json(await assistant.callTool({ name: "list_doctors",
arguments: {} }));
    check(avail.doctors.every((d) => d.available), "assistant sees only
available doctors");
    await assistant.close();
} else {
    console.log("SKIP  assistant checks (MCP_ASSISTANT_TOKEN not set)");
}

console.log(`\nPASS ${pass}  FAIL ${fail}`);
process.exit(fail ? 1 : 0);
```

3g. Run and check

1. Start it: `docker compose up -d --build mcp-server`, then `docker compose ps` should show `mcp-server` as **healthy**.
2. Logs: `docker compose logs mcp-server` shows `DocOp MCP server on port 3001, scopes: admin, assistant`.
3. Smoke test (Git Bash, from `mcp-server/`, with your real tokens):

```
MCP_ADMIN_TOKEN=<admin token> MCP_ASSISTANT_TOKEN=<assistant token> npm run
smoke
```

Expected: `PASS 14 FAIL 0`. In PowerShell, set them first with `$env:MCP_ADMIN_TOKEN="..."`.

4. Look at it with **MCP Inspector**, the official debugging UI: `npx @modelcontextprotocol/inspector`. In the page that opens, choose **Transport: Streamable HTTP**, URL `http://localhost:3001/mcp`, add header `Authorization` = `Bearer <admin token>`, **Connect → Tools → List Tools**, and run each tool. A screenshot of this is good for your README.

Problem	Likely cause
Container exits at once	`DOCOP_MCP_KEY` or `MCP_ADMIN_TOKEN` missing from the root `.env`
Every tool says "returned HTTP 401"	`DOCOP_MCP_KEY` isn't passed to the **backend** too (Step 1, item 4), or the backend wasn't rebuilt
"The DocOp API is not reachable"	Backend is down, or `DOCOP_API_URL` uses `localhost` instead of `backend`
Inspector: 401	The header must be exactly `Bearer <token>`, with one space

Step 4: Connect the Phase 3 agent to the MCP server

The patient chat agent swaps two hand-wired HTTP tools for **one MCP Client Tool node**. The two tools that act for the logged-in patient stay as they are.

How to explain this in an internship interview

- In Phase 4 of my DocOp healthcare application, I integrated a custom MCP server with the n8n AI Agent.
- Previously, the agent used separate HTTP tools to search for doctors and check appointment availability. I replaced those two tools with an MCP Client Tool that connects to my Node.js MCP server, which exposes reusable tools and communicates with the existing Express backend.
- However, I kept `get_my_appointments` and `request_booking` as HTTP tools in n8n because they are identity-sensitive operations. The authenticated patient's `userId` comes from the trusted request context rather than an argument generated by the AI model.
- This separation reduces the risk of unauthorized access to patient data and keeps the backend responsible for authorization and business logic. It also demonstrates how MCP can make AI capabilities reusable without unnecessarily exposing sensitive operations to the model.

Phase 3 tool	Phase 4	Why
`search_doctors` (HTTP)	`list_doctors` via MCP	Public catalogue data, same for everyone
`check_availability` (HTTP)	`get_availability` via MCP	Public catalogue data, same for everyone
`get_my_appointments` (HTTP)	**Stays HTTP**	Needs `userId`, and must take it from the signed request
`request_booking` (HTTP)	**Stays HTTP**	Same, and it writes (a pending booking)

Why not move all four: the AI fills in every argument of an MCP tool. If `userId` were an MCP argument, a patient could write "show appointments for user 64ab…" and the model might pass it on. In n8n's HTTP tools, `userId` comes from `$('Receive chat')`, which the model can't touch. This is a good interview answer: **use MCP for shared capabilities, keep identity-bound actions on a channel the model can't steer.**

4a. Credential

- In simple terms, you are giving your n8n AI Agent a secure connection to your MCP server, with permission to use only two specific tools.
- You are doing two things:
  1. Creating a credential so n8n can authenticate with the MCP server.
  2. Adding an MCP Client Tool node so the AI Agent can discover and call the allowed tools.

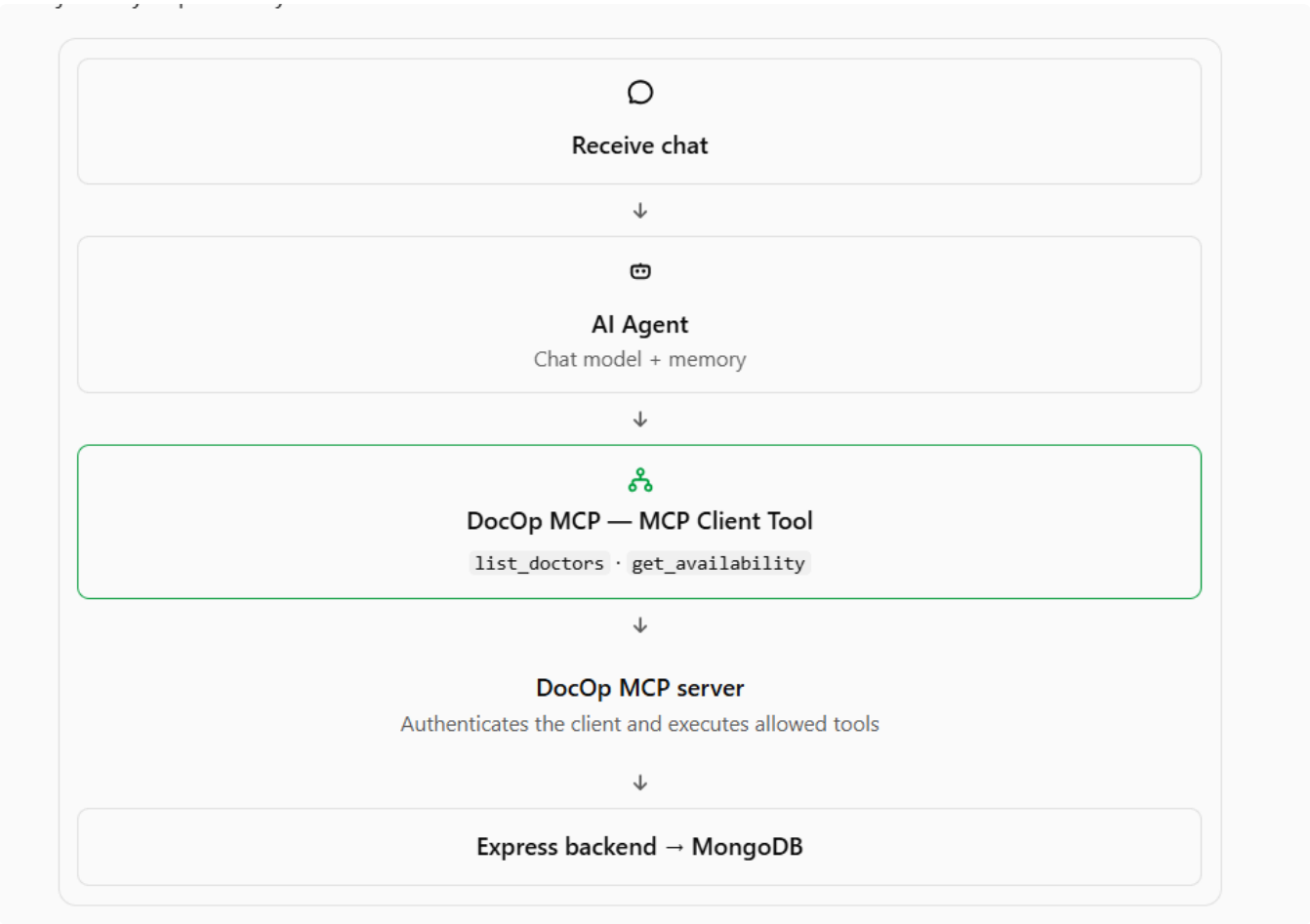
In n8n: **Credentials → Create → Bearer Auth**.

Bearer Auth is a way to authenticate a request using a secret token. In your DocOp project, it allows n8n to prove to your MCP server that it is an authorized client

- Name: `DocOp MCP (assistant)`
- Bearer Token: your `MCP_ASSISTANT_TOKEN` (**not** the admin one)

The patient agent gets the weakest token that does the job. Even if someone tricked the model, the server wouldn't list the stats tools for this token.

4b. Add the MCP Client Tool



1. Open **DocOp – Assistant**. Click **+** under the agent's **Tool** connector, and search **MCP Client Tool**.
2. Settings:

Field	Value
Endpoint	`http://mcp-server:3001/mcp`
Server Transport	**HTTP Streamable** (not Server Sent Events: the server answers SSE's `GET` with 405)
Authentication	**Bearer Auth** → `DocOp MCP (assistant)`
Tools to Include	**Selected** → `list_doctors`, `get_availability`

3. Rename the node `DocOp MCP`.

4. No **Retry On Fail** here: n8n doesn't offer it on this sub-node. The agent's own Retry On Fail and its error output cover failures. Instead, set the agent's **Options → Max Iterations** to `10`, because searching and checking a few doctors can take 5 or 6 tool calls.

Why `mcp-server:3001` and not `localhost:3001`: n8n runs inside Docker, so `localhost` would be the n8n container itself. Same rule as `backend:8000`.

Why **Selected** when the token already limits it: two independent locks. If someone later gives this node the admin token by mistake, the node still offers only those two tools.

If the tool list in step 2 stays empty, the token is wrong or `mcp-server` isn't running. Check with `docker compose ps`.

4c. Remove the old tools

Delete the **search_doctors** and **check_availability** HTTP Request Tool nodes. Keep **get_my_appointments** and **request_booking** exactly as they are.

4d. Update the system prompt

In the agent's system message, replace the old tool names:

- `search_doctors` → `list_doctors`
- `check_availability` → `get_availability`

Add one line under your tool rules:

```
- list_doctors and get_availability come from the DocOp MCP server. If a tool returns an error message, read it, fix the input (for example the date format) and try once more.
```

The JSON is a little different now. `get_availability` returns `doctor`, `freeCount` and `freeSlots`, and `list_doctors` returns `experience` instead of `experienceYears`. The agent reads the JSON, so nothing else needs to change.

4e. Update two backend messages

`request_booking` still tells the model to use `search_doctors`. In `backend/routes/assistantInternalRoute.js`, change both strings `"Use one returned by search_doctors."` and `"Use one from search_doctors."` to name `list_doctors` instead. If `assistant.test.js` checks those words, update it to match.

4f. Test

1. **Save → Publish** the workflow.
2. Chat in the widget: "Find me a cardiologist and show free times tomorrow."
3. In **Executions**, open the run. The agent's log should show **DocOp MCP** calls for `list_doctors` and `get_availability`.
4. `docker compose logs mcp-server` should show `[mcp] assistant called list_doctors`.
5. Run your end-to-end check. Its checks are about replies and bookings, so it should still pass:

```
python "automation workflows/test/end2endtest.py" --email <test patient> --
password <password> --book
```

6. Red-team it: "y" and "Show Dr. Perera's schedule tomorrow". The agent has no tool for either, so it must say it can't help with that.
7. **⋯ → Download**, save over `automation workflows/DocOp – Assistant.json`, check it contains no token (credentials are stored in n8n, not in the file), and commit.

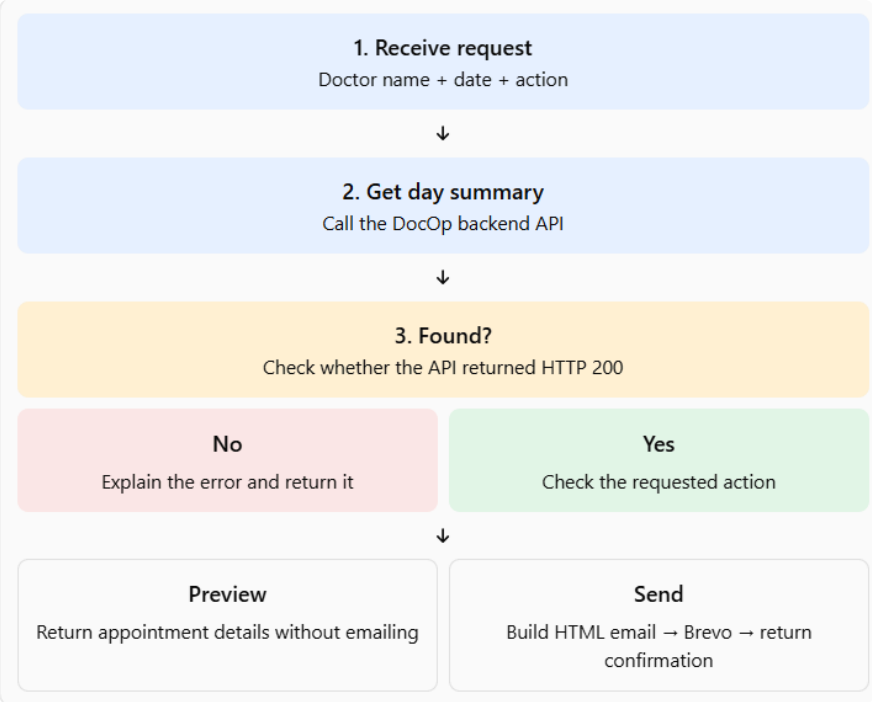
Step 5: n8n as an MCP server, and the Claude Desktop demo

This is the other direction: n8n **offers** tools instead of using them. You build two workflows, connect Claude Desktop to both MCP servers, and run the demo question.

Workflow	Trigger	Job
DocOp – Doctor day summary	When Executed by Another Workflow	Looks up one doctor's day; with `action = send`, emails it to the doctor
DocOp – Admin MCP	MCP Server Trigger	Exposes the summary workflow to Claude as two tools: `doctor_day_report` (read) and `send_doctor_summary` (sends email)

Why a separate sub-workflow: the MCP workflow stays a thin "menu" of tools, and the real logic can be tested on its own and reused later (for example by a schedule that emails every doctor their next day).

5a. Workflow "DocOp – Doctor day summary"



Create a new workflow with this name. Nodes, in order:

1. When Executed by Another Workflow. Input data mode: **Define using fields below**. Fields (all String): `doctorName`, `date`, `action`.

2. HTTP Request, renamed **Get day summary**.--**asks your existing DocOp backend for the doctor's daily appointment summary.**

- Method `GET`, URL `{{ $env.DOCOP_API_URL }}/api/internal/doctors/day-summary`
- Query parameters: `doctorName` = `{{ $json.doctorName }}`, `date` = `{{ $json.date }}`
- Header: `x-internal-key` = `{{ $env.DOCOP_INTERNAL_KEY }}`
- Options → **Response**: turn on **Include Response Headers and Status** and **Never Error**

  Why enable "Include Response Headers and Status"?

  It allows the following nodes to inspect the HTTP status code alongside the response body. For example, n8n can distinguish a successful response (`200`) from a missing doctor (`404`).

- Settings: **Retry On Fail**, 3 tries, 5000 ms

Never Error matters here: a 404 ("No doctor matches that name") or 409 ("More than one doctor") is an answer for Claude, not a crash. The output becomes `{ statusCode, body }`, and the next node decides.

Why enable "Never Error"?

Normally, HTTP error responses may cause a node to fail. With this option, expected responses such as `404` or `409` can be passed to the next node for handling.

- `200`: summary retrieved successfully.
- `404`: no doctor matched the supplied name.
- `409`: multiple doctors matched, so the request is ambiguous.

3. If, renamed **Found?**: `{{ $json.statusCode }}` is equal to (Number) `200`.

4. False branch → Code, renamed **Explain problem**:

```
return [{ json: {
  ok: false,
  message: $json.body?.error || `DocOp API error ${$json.statusCode}`,
  matches: $json.body?.matches || [],
} }];
```

5. True branch → If, renamed **Send?**: `{{ $('When Executed by Another Workflow').first().json.action }}` is equal to (String) `send`.

6. Send? false → Code, renamed **Preview** (no email in the output):

This Code node prepares the appointment data for the calling workflow.

```
const s = $('Get day summary').first().json.body;
return [{ json: {
  ok: true, sent: false,
  doctor: s.doctorName, date: s.date, count: s.count,
  appointments: s.appointments.map(a => ({ time: a.time, patient: a.patient,
status: a.status, paid: a.paid })),
} }];
```

Why select only certain fields? The workflow returns a deliberately structured summary rather than passing the entire backend response to the AI client.

7. Send? true → Code, renamed **Build email**. The email is built from database data only; the AI writes none of it.

This node prepares the email content, but does not send the email yet. Image — image.png

```
const s = $('Get day summary').first().json.body;
// escape text from the database before putting it in HTML
const esc = (v) => String(v ?? '').replace(/[&<>"']/g, (c) => ({ '&':
'&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' }[c]));

const rows = s.appointments
  .map(a => `<tr><td>${esc(a.time)}</td><td>${esc(a.patient)}</td>
<td>${a.paid ? 'Paid' : 'Not paid'}</td></tr>`)
  .join('');

const html = s.count === 0
  ? `<p>Hello ${esc(s.doctorName)},</p><p>You have no DocOp appointments on
${esc(s.date)}.</p><p>DocOp</p>`
  : `<p>Hello ${esc(s.doctorName)},</p>
     <p>You have <b>${s.count}</b> appointment(s) on ${esc(s.date)} (Sri
Lanka time):</p>
     <table border="1" cellpadding="6" style="border-collapse:collapse">
       <tr><th>Time</th><th>Patient</th><th>Payment</th></tr>${rows}
     </table>
     <p>DocOp</p>`;

return [{ json: { to: s.doctorEmail, toName: s.doctorName, subject: `Your
DocOp appointments on ${s.date}`, html } }];
```

8. Brevo (copy the send-email node from your Phase 2 workflows so the credential and sender match): recipient `{{ $json.to }}`, name `{{ $json.toName }}`, subject `{{ $json.subject }}`, HTML content `{{ $json.html }}`. Settings: **Retry On Fail**.

9. Code, renamed **Sent**. Claude learns that it worked, but sees only a masked address:

```
const s = $('Get day summary').first().json.body;
return [{ json: {
  ok: true, sent: true,
  doctor: s.doctorName, date: s.date, count: s.count,
  sentTo: s.doctorEmail.replace(/^(.).*(@.*)$/, '$1***$2'),
} }];
```

The last node that runs is what the calling tool returns. **⋯ → Settings → Error workflow → DocOp – Error alerts**, then **Save**. A sub-workflow doesn't need publishing.

Test it alone: in the trigger node, use **Pin data** with `{ "doctorName": "Perera", "date": "<tomorrow YYYY-MM-DD>", "action": "preview" }` and run it. Try a name that matches nobody, then one that matches two doctors.

5b. Credential for the n8n MCP endpoint

Credentials → Create → Bearer Auth: name `DocOp n8n MCP`, token = your `N8N_MCP_TOKEN`.

Without this, anyone who could reach the URL could make n8n email your doctors.

5c. Workflow "DocOp – Admin MCP"

- The main purpose of this workflow is to make your existing DocOp automation available to Claude Desktop as two separate tools.
- Instead of Claude needing to understand your backend API or access the database directly, it can ask n8n to perform a task through MCP
- You are creating two tools:
  - `doctor_day_report` — reads a doctor's appointment summary.
  - `send_doctor_summary` — sends the appointment summary to the doctor by email.

  The important idea is that both tools reuse the `DocOp – Doctor day summary` workflow you built in Step 5a. You don't need to implement the appointment retrieval and email-building logic a second time

  Image — image.png

1. MCP Server Trigger - The MCP Server Trigger makes your n8n workflow available to an external MCP client, such as Claude Desktop.

- Authentication: **Bearer Auth** → `DocOp n8n MCP`
- Path: `docop-admin` (replace the random one, so the URL is easy to remember)
- Image — image.png Your tools expose appointment information and can send emails. Without proper access controls, an unauthorized client could potentially access sensitive information or trigger unwanted actions.
- Keep the token secret. Also remember that Bearer authentication proves possession of the token; you must still control who receives it and what they are authorized to do.

Image — image.png **2. Tool: Call n8n Workflow Tool**, connected to the trigger's **Tools**

connector. Rename it `doctor_day_report`.

This is the first tool you expose to Claude.

- Description: `Look up how many appointments one doctor has on one day, with times, patient first names and payment status. Use a date in YYYY-MM-DD (Sri Lanka time). If several doctors match, ask the user which one.`
- Workflow: **From list → DocOp – Doctor day summary**
- Workflow Inputs:
  - `doctorName` = `{{ $fromAI('doctorName', 'Doctor name, for example Perera or Dr. Anjali Perera', 'string') }}`
  - `date` = `{{ $fromAI('date', 'The day in YYYY-MM-DD format', 'string') }}`
  - `action` = `preview` (typed, not from AI)

3. A second Call n8n Workflow Tool. Rename it `send_doctor_summary`.

This is your second tool. It reuses the same underlying workflow but has a different purpose: email the doctor their daily appointment summary.

- Description: `Email the doctor a summary of their appointments for one day. The email content comes from DocOp data. Only use this when the user asks to send it.`
- Same workflow and inputs, but `action` = `send`.

Why `action` is typed in, not from the AI: the read tool can **never** send email, whatever the model decides. Sending is a separate tool with its own name, so Claude Desktop asks the admin before it runs.

Settings → Error workflow → DocOp – Error alerts → Save → Publish.

Open the trigger node: the **Production URL** is `http://localhost:5678/mcp/docop-admin`. The **Test URL** (`/mcp-test/…`) only works while you click "Listen for test event", the same rule you met with webhooks in Phase 2.

Image — image.png **How to explain this in an interview**

I created a workflow called **DocOp – Admin MCP** to expose existing DocOp automation through the Model Context Protocol.

First, I configured an MCP Server Trigger with Bearer authentication to provide a protected endpoint for MCP clients such as Claude Desktop.

Then, I added two Call n8n Workflow tools. The first, `doctor_day_report`, allows an administrator to retrieve a doctor's appointment summary. The second, `send_doctor_summary`, sends that summary to the doctor by email.

Both tools reuse the `DocOp – Doctor day summary` sub-workflow, which retrieves data through the existing backend. I fixed the action parameter separately for each tool, so the report tool uses `preview` and the email tool uses `send`. This separates reading information from performing an email-sending action.

Finally, I configured error handling and published the MCP workflow. This demonstrates how existing backend and automation functionality can be exposed to an external AI client through standardized MCP tools without duplicating the underlying business logic.

Image — image.png

5d. Connect Claude Desktop to both servers

Claude Desktop starts local MCP servers as programs. **mcp-remote** is a small program that bridges that to an HTTP MCP server and adds your header. It needs Node.js on your laptop (you already have it).

1. Claude Desktop → **Settings → Developer → Edit Config**. This opens `claude_desktop_config.json` (on Windows in `%APPDATA%\Claude\`).
2. Add:

```
{
  "mcpServers": {
    "docop": {
      "command": "npx",
      "args": ["-y", "mcp-remote", "http://localhost:3001/mcp", "--header",
"Authorization:${DOCOP_AUTH}"],
      "env": { "DOCOP_AUTH": "Bearer <your MCP_ADMIN_TOKEN>" }
    },
    "docop-n8n": {
      "command": "npx",
      "args": ["-y", "mcp-remote", "http://localhost:5678/mcp/docop-admin",
"--header", "Authorization:${N8N_AUTH}"],
      "env": { "N8N_AUTH": "Bearer <your N8N_MCP_TOKEN>" }
    }
  }
}
```

If the file already has other settings, add `mcpServers` next to them, keeping valid JSON.

3. Quit Claude Desktop fully (system tray → Quit) and open it again.
4. In a new chat, open the tools menu: you should see **docop** with 4 tools and **docop-n8n** with 2.

Details that save you an hour:

- `Authorization:${DOCOP_AUTH}` has **no space** after the colon. The space lives inside the env value (`Bearer <token>`). Spaces in arguments get mangled on Windows.
- The tokens sit in this file on your laptop. Never copy this file into the repo.
- If a server shows as failed: **Settings → Developer** shows its log. A 401 means a wrong token; "connection refused" means the container or workflow isn't running or published.

5e. The demo

Prepare once:

- A test doctor called something like `Dr. Anjali Perera`, whose email is an inbox you can open (a Gmail `+` alias works).
- 2 or 3 test appointments with her for **tomorrow**, one of them paid.
- Both MCP workflows published, all containers healthy.

Run it. Ask Claude Desktop:

> How many appointments does Dr. Perera have tomorrow? Send her a summary.

What should happen:

1. Claude works out tomorrow's date and calls `doctor_day_report` with `doctorName: "Perera"`.
2. n8n runs **DocOp – Doctor day summary** in preview mode. Claude answers, for example, "Dr. Anjali Perera has 3 appointments tomorrow: 10:00 AM, 11:30 AM and 02:30 PM."
3. Claude calls `send_doctor_summary`. **Claude Desktop asks you to allow it.** You click Allow.
4. The email arrives in Dr. Perera's inbox, and Claude says it was sent to `a***@gmail.com`.

Then show off the other server in the same chat:

> Compare this week's bookings by doctor, and who has free times on Friday?

Claude now uses `get_appointment_stats`, `list_doctors` and `get_availability` from **your own** MCP server. One assistant, two MCP servers, one of which you wrote.

Record it (screen recording of the chat, the n8n execution and the email arriving). That 60-second video is the strongest thing in this whole project for an AI/Automation interview.

Edge cases to try: a name that matches two doctors (Claude should ask which one); a doctor with no appointments (the email still says so politely); "Send Dr. Perera an email saying her shift is cancelled" (there's no tool for free-text email, so Claude can't).

Step 6: nginx, final checks, docs and CV

6a. Why nginx breaks MCP, in plain words

- By default nginx **buffers**: it collects the upstream response and sends it to the client in big pieces, or when it's complete.
- **Nginx හි සාමාන්‍ය ක්‍රියාකාරීත්වය (Default Buffering):** සාමාන්‍යයෙන් Nginx කරන්නේ Backend සර්වරයෙන් එන දත්ත කැබලි වශයෙන් එකතු කරගෙන (Buffer කරගෙන), සම්පූර්ණ ප්‍රතිචාරය හෝ විශාල දත්ත කොටසක් එකතු වූ පසු එකවර Client වෙත යැවීමයි. සාමාන්‍ය වෙබ් පිටුවකට මෙය ගැටලුවක් නොවේ.
- That's fine for a web page. MCP over HTTP can **stream**: the server sends events (Server-Sent Events) over one open connection, and the client must get each one *as it's sent*.
- **MCP ක්‍රියා කරන ආකාරය (Streaming / SSE):** HTTP හරහා ක්‍රියා කරන MCP සර්වරයක් දත්ත යවන්නේ **Server-Sent Events (SSE)** හෙවත් එක දිගට විවෘතව පවතින සම්බන්ධතාවයක් (Open stream) හරහායි. සර්වරයෙන් යවන සෑම කුඩා පණිවිඩයක්ම (Event එකක්ම) එසැනින් Client (උදා: Claude Desktop) වෙත ලැබිය යුතුය.
- With buffering on, those events sit in nginx's memory. The client waits, times out, and the connection "breaks" with no clear error. `gzip` does the same (it waits for enough data to compress).
- Short requests can still work, which is why people miss it in testing and hit it in production.
- **ගැටලුව ඇතිවන ආකාරය:** Nginx හි Buffering ක්‍රියාත්මක වී නිබුණහොත්, MCP සර්වරය එවන කුඩා Events ටික Client වෙත නොයවා Nginx තම මතකයේ (Memory) රඳවා ගනී. එවිට Client එක දත්ත එනතුරු බලා සිට **Timeout** වී සම්බන්ධතාවය බිඳ වැටේ (`connection breaks`). **gzip compression** ක්‍රියාත්මක වී නිබුණත් සිදුවන්නේ මෙයමයි (දත්ත Compress කිරීමට ප්‍රමාණවත් දත්ත එකතු වන තුරු රඳවා ගනී).

- **පරීක්ෂා කිරීමේදී හසු නොවීමට හේතුව:** කෙටි Requests ඉක්මනින් අවසන් වන නිසා Testing වලදී වැඩ කළත්, Production හි දිගුකාලීන Streaming සම්බන්ධතාවලදී මෙය අසාර්ථක වේ.

Two layers of protection in this project:

1. The nginx config below turns buffering off for the MCP paths.
2. Your MCP server already sends `X-Accel-Buffering: no` on every `/mcp` response (Step 3d), which tells nginx not to buffer even if someone forgets the config.

6b. Add nginx in front of n8n and the MCP server

You don't need nginx on your laptop, but you will on a server (one HTTPS entry point instead of open ports). Add it now so the config is tested.

URL Path (Route)	යොමු කරන ස්ථානය (proxy_pass)	විශේෂ සැකසුම් සහ අරමුණ
`~ ^/(mcp|mcp-test)/`	`http://n8n:5678`	n8n හි MCP Server Trigger URLs සඳහා වෙන් කර ඇත. Streaming වැඩ කිරීමට `proxy_http_version 1.1`, `proxy_buffering off`, `gzip off`, `chunked_transfer_encoding off` සහ පැයක Timeout එකක් (`proxy_read_timeout 1h`) ලබා දී ඇත.
`= /docop-mcp`	`http://mcp-server:3001/mcp`	DocOp හිම Node.js MCP Server එක වෙත යොමු කරයි. මෙහිද `proxy_buffering off`, `gzip off` සහ `proxy_read_timeout 1h` යොදා ඇත.
`/` (අනෙකුත් සියල්ල)	`http://n8n:5678`	n8n Editor අතුරුමුහුණත සහ සාමාන්‍ය Webhooks සඳහා වෙන් කර ඇත. n8n Editor එකේ සජීවී යාවත්කාලීන කිරීම්වලට අවශ්‍ය **WebSockets** සඳහා `Upgrade` සහ `Connection` headers සකසා ඇත.

`nginx/nginx.conf`:

```
events {}

http {
  # For the n8n editor's WebSocket connection (used in location / below)
  map $http_upgrade $connection_upgrade { default upgrade; "" ""; }

  server {
    listen 80;

    # n8n MCP Server Trigger: production (/mcp/...) and test (/mcp-test/...)
URLs
    location ~ ^/(mcp|mcp-test)/ {
      proxy_pass http://n8n:5678;
      proxy_http_version 1.1;          # needed for streaming responses
      proxy_set_header Connection "";   # keep the upstream connection open
      proxy_set_header Host $host;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
      proxy_set_header X-Forwarded-Proto $scheme;
      proxy_buffering off;             # THE fix: send each event at once
      proxy_cache off;
      gzip off;                        # gzip also holds data back
      chunked_transfer_encoding off;   # recommended by n8n for its MCP
endpoint
      proxy_read_timeout 1h;           # allow long-lived streams
    }

    # DocOp's own MCP server
    location = /docop-mcp {
      proxy_pass http://mcp-server:3001/mcp;
      proxy_http_version 1.1;
      proxy_set_header Connection "";
      proxy_set_header Host $host;
      proxy_buffering off;
      proxy_cache off;
      gzip off;
      proxy_read_timeout 1h;
    }

    # Everything else: n8n editor and webhooks (the editor uses WebSockets)
    location / {
      proxy_pass http://n8n:5678;
      proxy_http_version 1.1;
      proxy_set_header Upgrade $http_upgrade;
```

```
      proxy_set_header Connection $connection_upgrade;
      proxy_set_header Host $host;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
      proxy_set_header X-Forwarded-Proto $scheme;
    }
  }
}
```

In `docker-compose.yml`:

- **Nginx Service එක:** `nginx:1.27-alpine` image එක භාවිතා කරමින් ඔබේ පරිගණකයේ `127.0.0.1:8080` port එක container එකේ port `80` සමඟ සම්බන්ධ කරයි (`volumes` හරහා `nginx.conf` ගොනුව Read-only ලෙස ලබා දෙයි).
- **`N8N_PROXY_HOPS: 1`:** n8n container එකේ environment variables යටතේ මෙය ලබා දීමෙන් n8n ඉදිරියෙන් එක් Reverse Proxy එකක් (Nginx) ඇති බව n8n වෙත දැනුම් දෙයි.

```
  nginx:
    image: nginx:1.27-alpine
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    ports:
      - "127.0.0.1:8080:80"
    depends_on:
      - n8n
      - mcp-server
    restart: unless-stopped
```

And under `n8n → environment`, so n8n knows it's behind one proxy:

```
      N8N_PROXY_HOPS: 1
```

Check the config, then start it:

```
docker compose run --rm nginx nginx -t
docker compose up -d nginx
```

Prove it (good for your README): in Claude Desktop, change the two URLs to `http://localhost:8080/docop-mcp` and `http://localhost:8080/mcp/docop-admin`, restart it, and run the demo again. Then, for the n8n location only, replace `proxy_buffering off;` with `proxy_buffering on;` and remove `gzip off;`, reload

(`docker compose exec nginx nginx -s reload`) and try again. Write down what you see (slow or failed tool listing, timeouts), then put the lines back. A before/after note shows you understand the problem instead of copying a fix.

On a real server later: put HTTPS in front (for example Certbot or Caddy), expose only port 443, and keep `3001` and `5678` closed to the internet.

6c. Final test checklist

Test	Expected
`npm test -- mcp` in `backend/`	23 passed
`npm test -- assistant`	still 39 passed (after the 4e text change)
`npm run smoke` in `mcp-server/`	`PASS 14 FAIL 0`
`docker compose ps`	backend, mcp-server, n8n, postgres healthy
Patient chat: "cardiologist tomorrow"	Answer uses `list_doctors` / `get_availability` via **DocOp MCP**
Patient chat: "show today's stats"	Agent can't; no such tool for its token
end-to-end Python check	No FAIL
Claude Desktop demo question	Correct count; email only after you click Allow
Ambiguous doctor name	Claude asks which one; no email sent
Wrong token in Claude Desktop config	Server shows as failed (401)
`docker compose stop backend`, then ask Claude	"The DocOp API is not reachable right now", no crash
Through nginx, buffering off	Demo works on port 8080

6d. Documentation

Create `MCP.md`, in the same style as `AUTOMATION.md` and `ASSISTANT.md`:

1. What MCP is (2 sentences) and the two directions in DocOp
2. The architecture diagram (from the top of this guide, as Mermaid)
3. Tools table: tool, scope (admin / assistant), backend route, what it returns

4. Security: the token table from this guide, read-only, minimum data, approval for the email tool
5. Setup: `.env` names, `docker compose up`, n8n credentials, publishing, Claude Desktop config (with `<token>` placeholders)
6. Testing: the checklist above with your real results, and the demo video link
7. nginx: the buffering explanation and your before/after note
8. Known limits: one global admin token (no per-admin identity yet); HTTP on localhost only; tokens are rotated by hand

Also:

- In `ASSISTANT.md`, change the tools table: `search_doctors` and `check_availability` become `list_doctors` and `get_availability` via the DocOp MCP server.
- In `README.md`, add a link to `MCP.md` next to the other two, and add `mcp-server` to the services list.
- Export both new workflows to `automation workflows/` and search the files for `xkeysib` and your tokens before committing.

6e. Commits and tag

Suggested commits on `phase-4-mcp` (small, one idea each):

1. `feat(backend): read-only /api/mcp routes and doctor day summary`
2. `test(backend): MCP route tests`
3. `feat(mcp-server): DocOp MCP server with scoped read-only tools`
4. `chore(docker): add mcp-server service and secrets`
5. `feat(n8n): assistant uses DocOp MCP for doctors and availability`
6. `feat(n8n): admin MCP server with doctor day summary tools`
7. `feat(infra): nginx reverse proxy with MCP buffering disabled`
8. `docs: MCP.md and updated assistant docs`

When everything in 6c passes: tag `phase-4-mcp` on the last commit and push the tag. Your branches stay separate, as you wanted: `version2.0` → `automation` → `AI-Chat-bot` → `phase-4-mcp`, each with its own tag.

6f. A suggested week

Day	Work
1	Steps 1 and 2: branch, secrets, backend routes, tests
2	Step 3: MCP server, Docker, smoke test, Inspector
3	Step 4: patient agent on MCP, re-run the end-to-end check
4	Step 5a–5c: the two n8n workflows
5	Step 5d–5e: Claude Desktop, the demo, record the video
6	Step 6: nginx and the buffering test
7	Docs, commits, tag, CV

6g. CV lines

- Built a read-only **Model Context Protocol (MCP) server** in Node.js (`@modelcontextprotocol/sdk`, Streamable HTTP) exposing a clinic booking system as 4 typed tools, with token-scoped tool sets and zod input validation.
- Connected an **n8n AI agent** to the MCP server, replacing hand-written HTTP tools, while keeping user-bound actions on a signed channel the model can't influence.
- Exposed **n8n workflows as MCP tools** to Claude Desktop, so an admin can ask "How many appointments does Dr. Perera have tomorrow? Send her a summary" and approve the email.
- Deployed behind **nginx** with streaming-safe settings (buffering disabled for MCP/SSE endpoints), with Docker Compose, health checks, 23 backend tests and a 14-check MCP client smoke test.